Ensemble Classification

2026 Spring

Industrial Data Science Lab & Unique AI

1. Classification Review

➢ Machine Learning 모형 구분

지도학습 (Supervised Learning)

종속 및 독립변수를 이용하여 주어진 독립(설명)변수를 바탕으로 종속(반응)변수 예측 모형 제시

예: 회귀/분류 모형

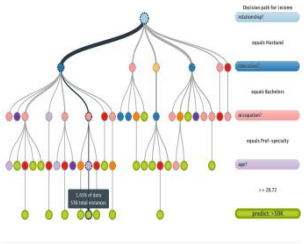
+

비지도학습 (Unsupervised Learning)

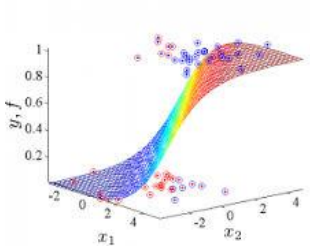
Target(종속변수/반응변수)이 없으며, 독립(설명)변수 간의 관계나 이를 바탕으로 개체들을 구분하여 의미 있는 결과를 제시

예:군집 분석, 연관성 분석, 주성분 / 요인분석

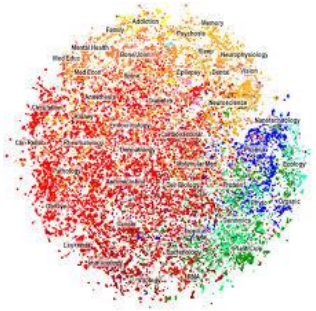
[decision tree]



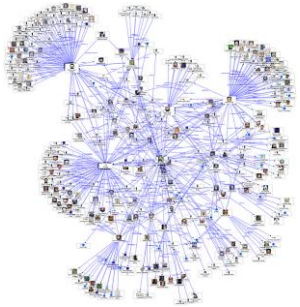
[logistic regression]



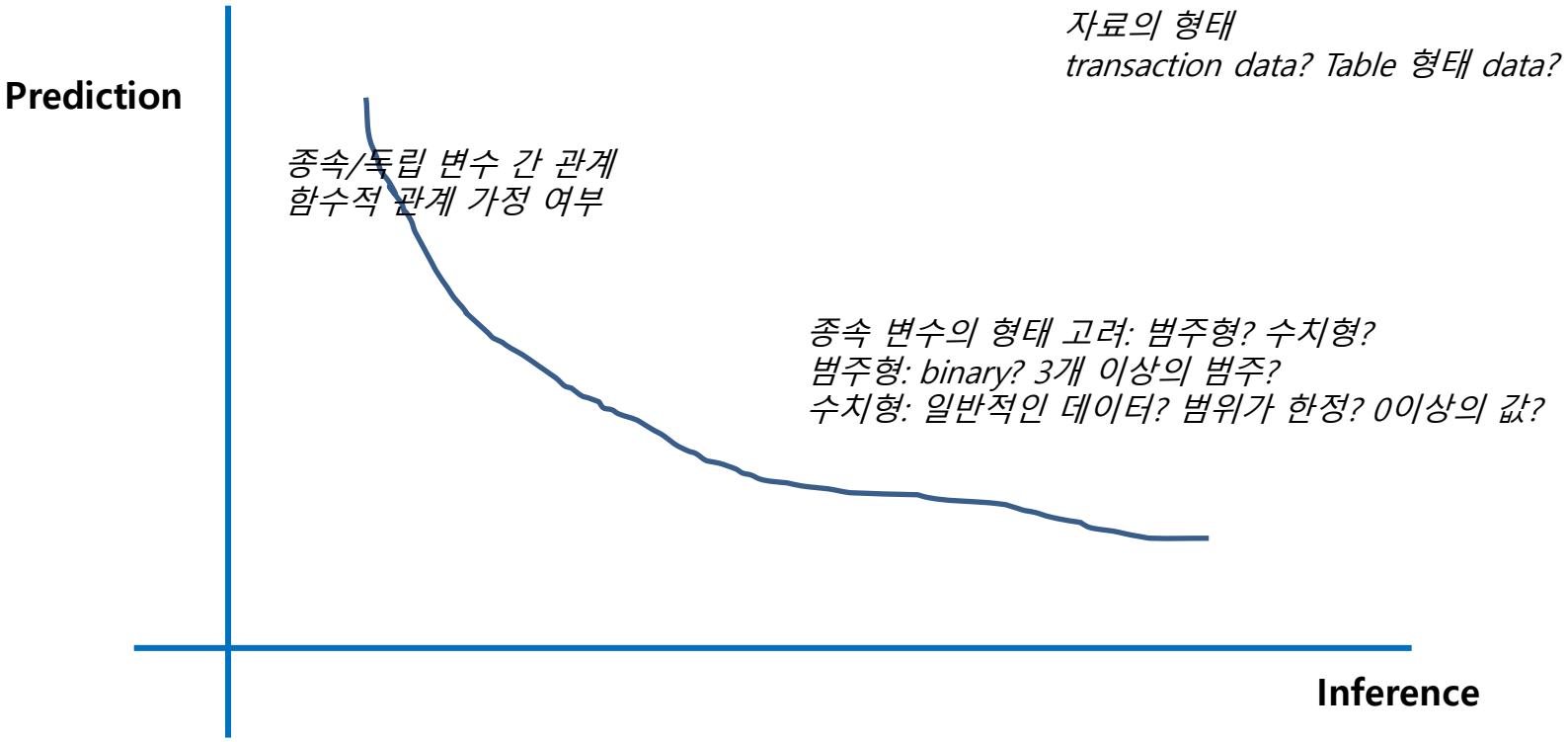
[clustering analysis]



[link analysis]



➢ 데이터 분석의 목적


Prediction
Inference
자료의 형태
transaction data? Table 형태 data?
종속/독립 변수 간 관계
함수적 관계 가정 여부
종속 변수의 형태 고려: 범주형? 수치형?
범주형: binary? 3개 이상의 범주?
수치형: 일반적인 데이터? 범위가 한정? 0이상의 값?

1. Classification Review

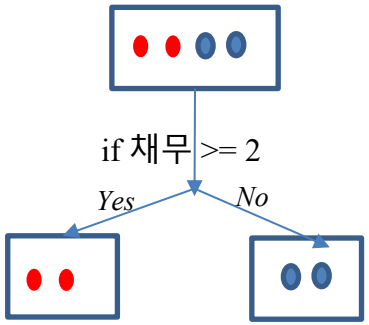
- **분류모형의 평가**
  - Confusion Matrix

	실제 Y	실제 N
예측 Y	True Positive(TP)	False Positive(FP)
예측 N	False Negative(FN)	True Negative(TN)

  - N=TP+FP+FN+TN
  - 예측 결과에 따라 True, False 구분
  - 예측 값에 따라 Positive, Negative 구분

Metric	Formula	설명
정분류율 or Accuracy	(TP+TN)/N	전체 결과 중 맞게 분류한 비율
오분류율	(FP+FN)/N	전체 결과 중 잘못 분류한 비율
Precision	TP/(TP+FP)	Y로 예측된 것 중 실제로도 Y인 비율

1. Classification Review



- Tree 구조: 그래프의 일종, Root로 부터 시작해서, 부모 노드에서 자식노드로 분기해나가며 구성된 자료 구조

Decision Tree(의사결정나무)
"일련의 간단한 규칙으로 만들어진, Target을 나눠가는 Tree 구조"

1. Classification Review

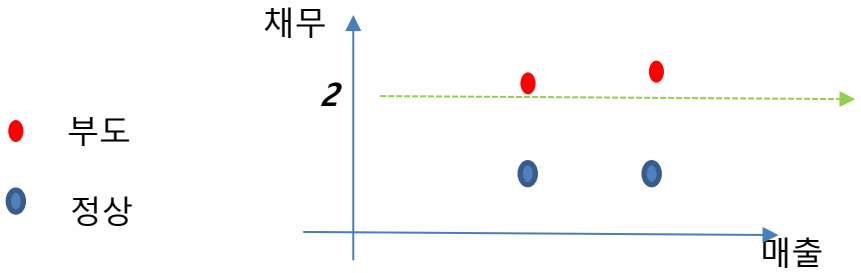
회사	부도여부	채무	매출
AAA	N	1	5.5
BBB	N	1	4.5
CCC	Y	2	4.5
DDD	Y	2.1	5.5

채무 변수의 값들: 1, 2, 2.1

1을 기준으로
1 이상 VS 1 미만

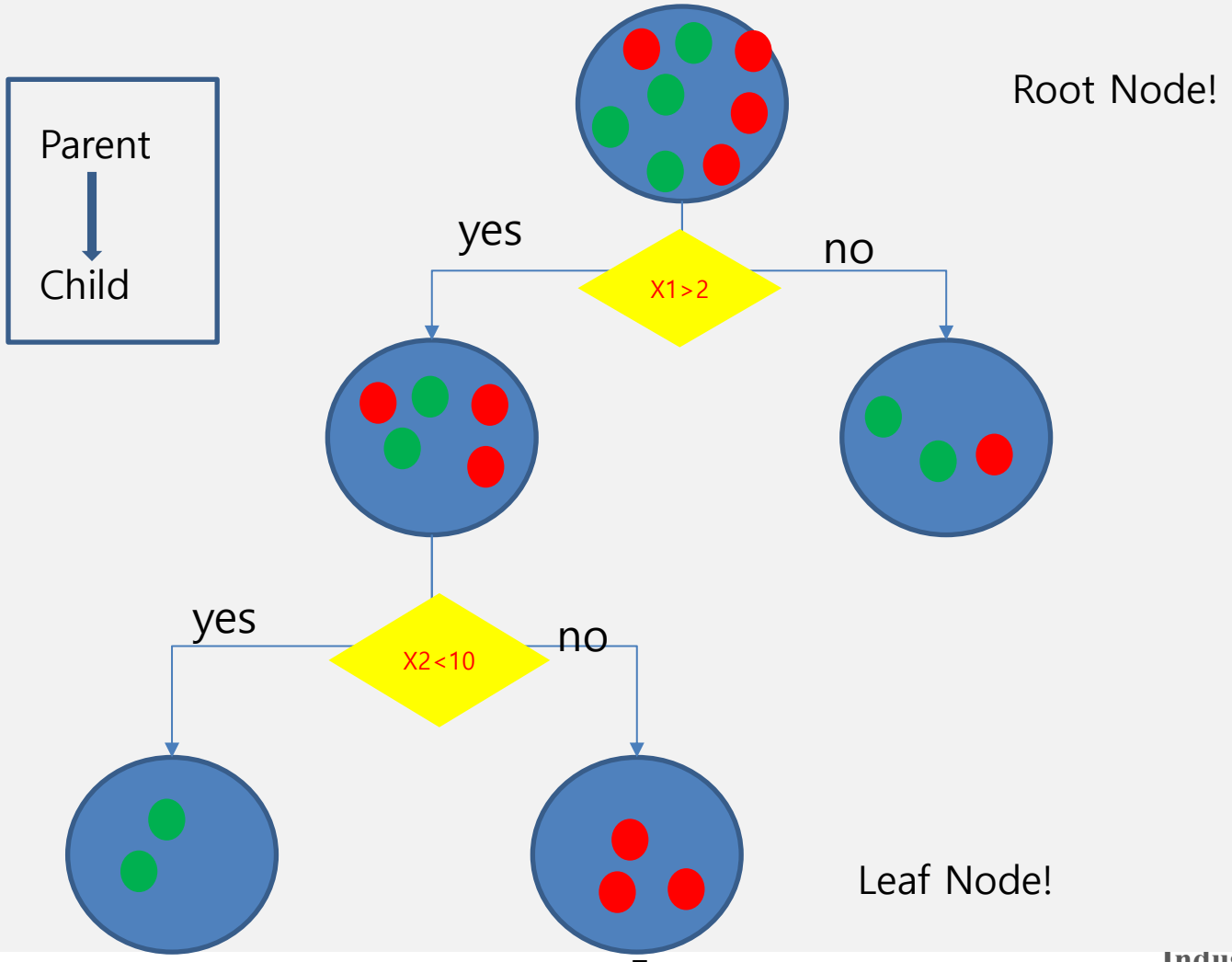
2를 기준으로
2 이상 VS 2 미만

2.1을 기준으로
2.1 이상 VS 2.1 미만

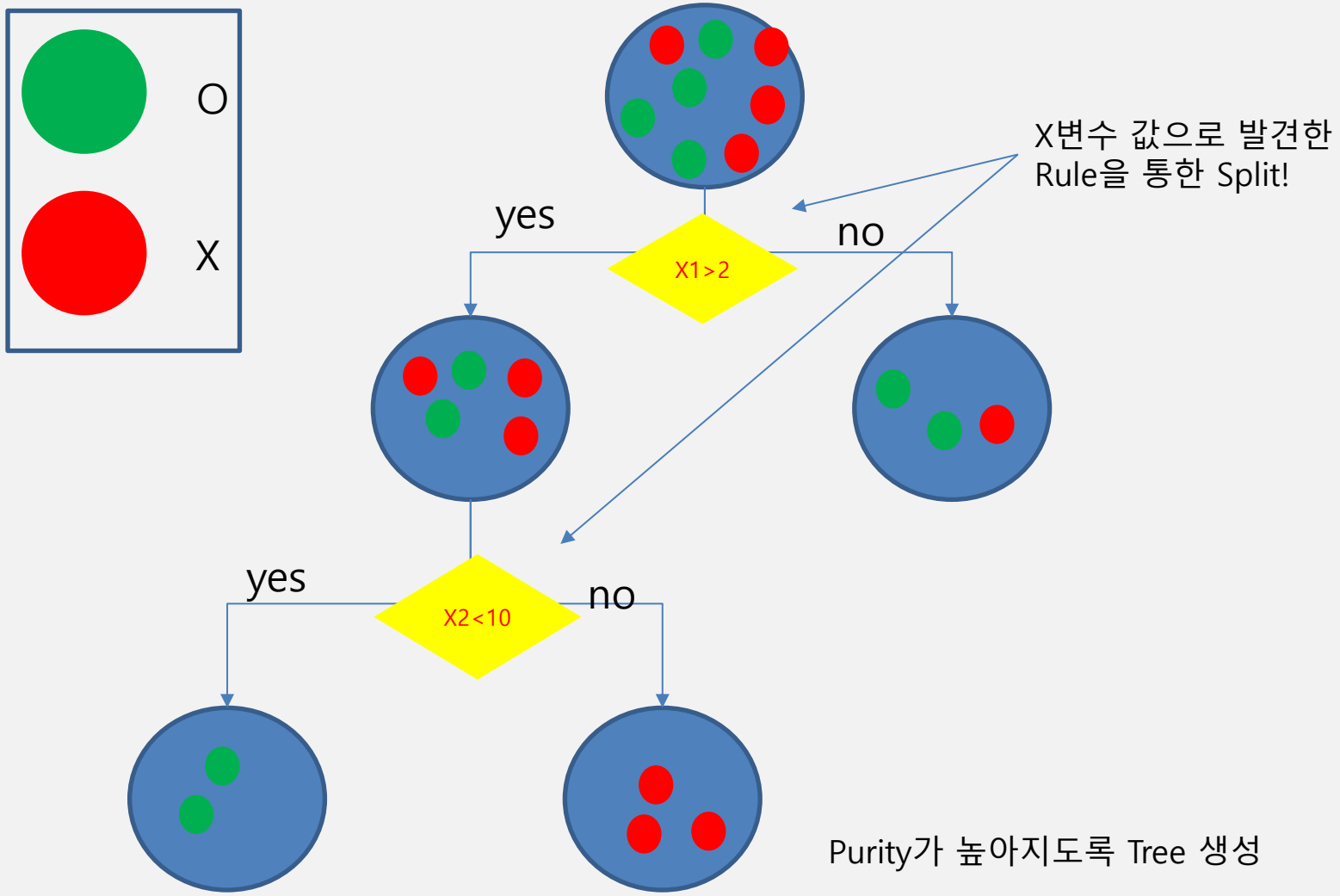


1. Classification Review

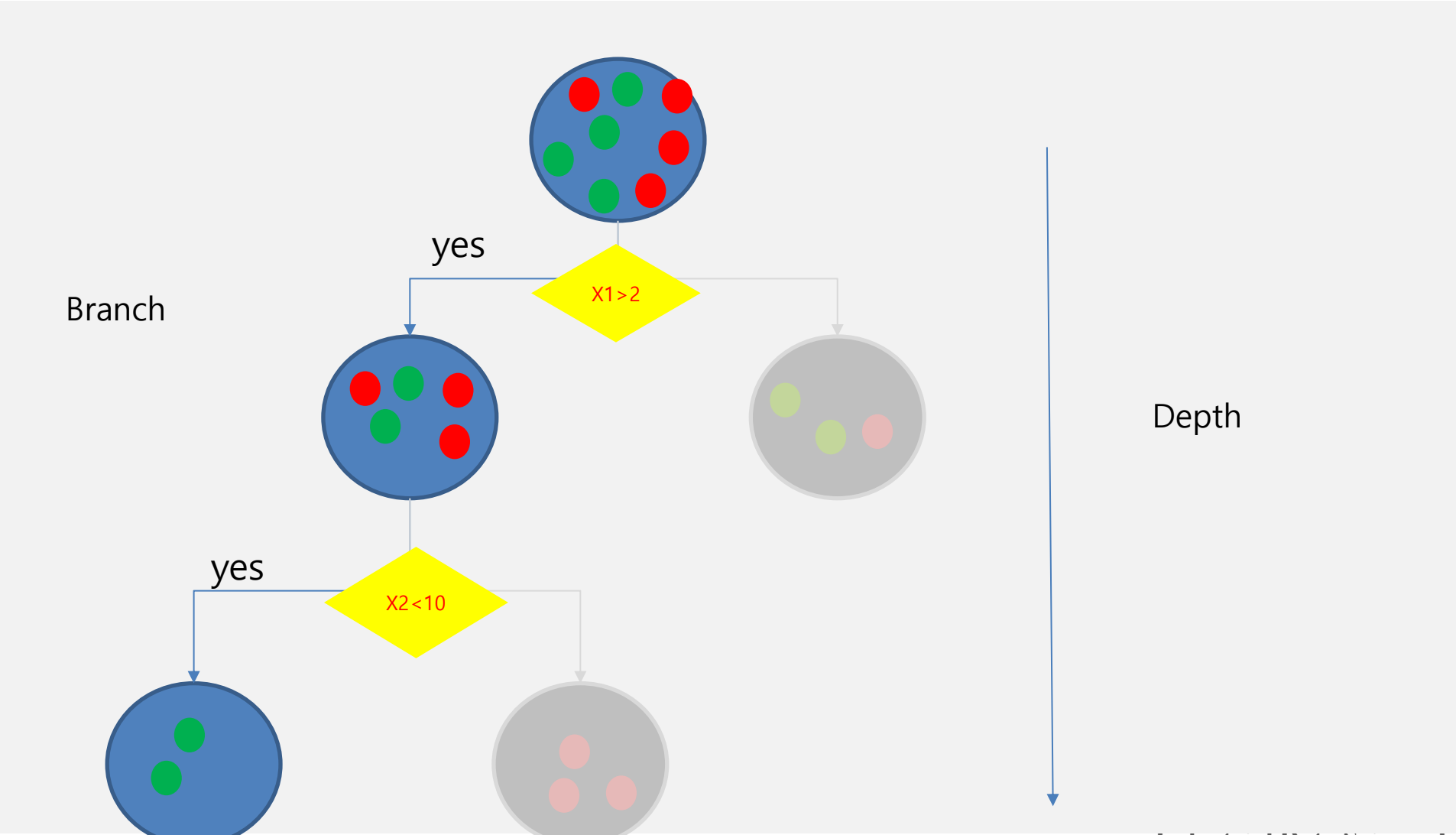
Decision Tree의 구성요소: Roof, Leaf Node



Decision Tree의 구성요소: Rule



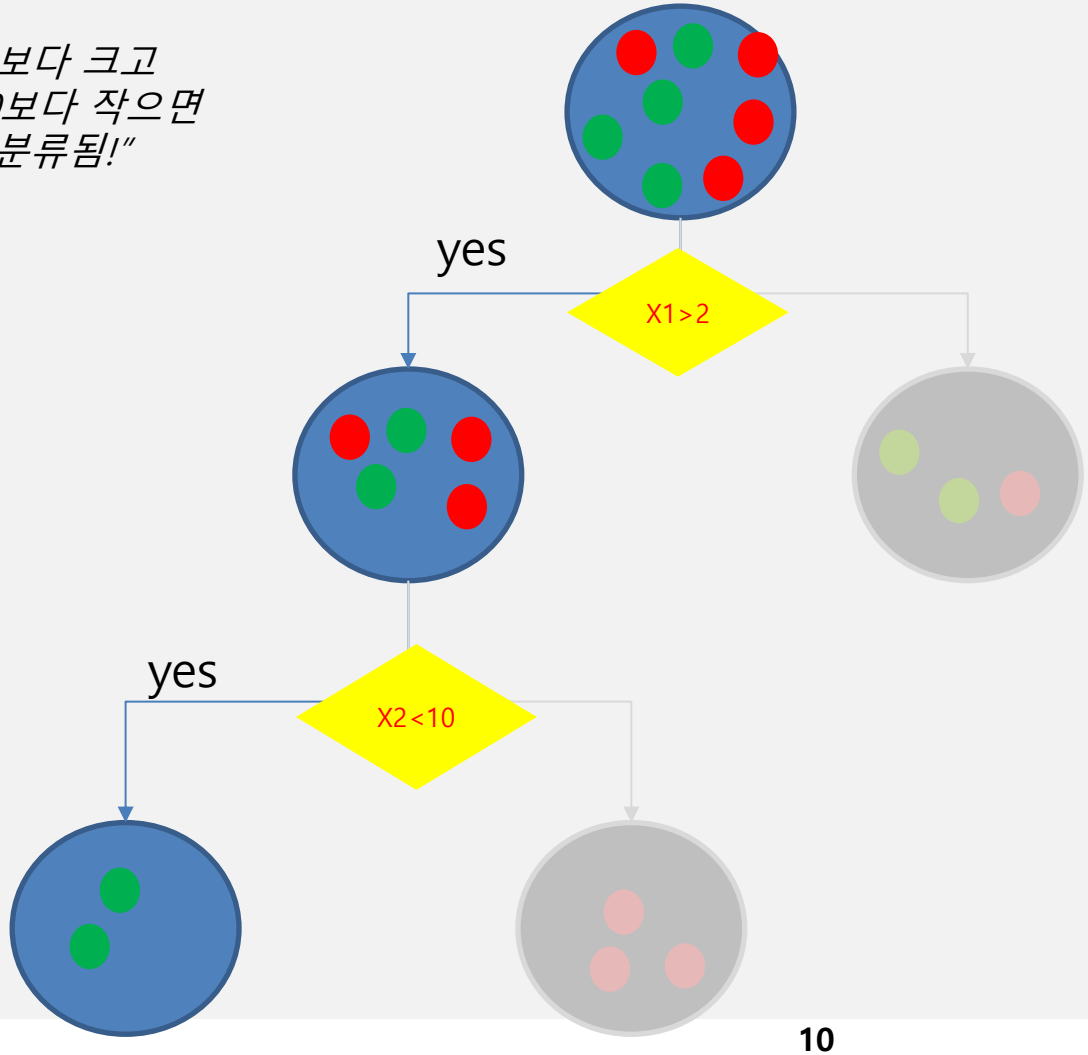
Decision Tree의 구성요소: Branch & Depth



1. Classification Review

Decision Tree의 이해

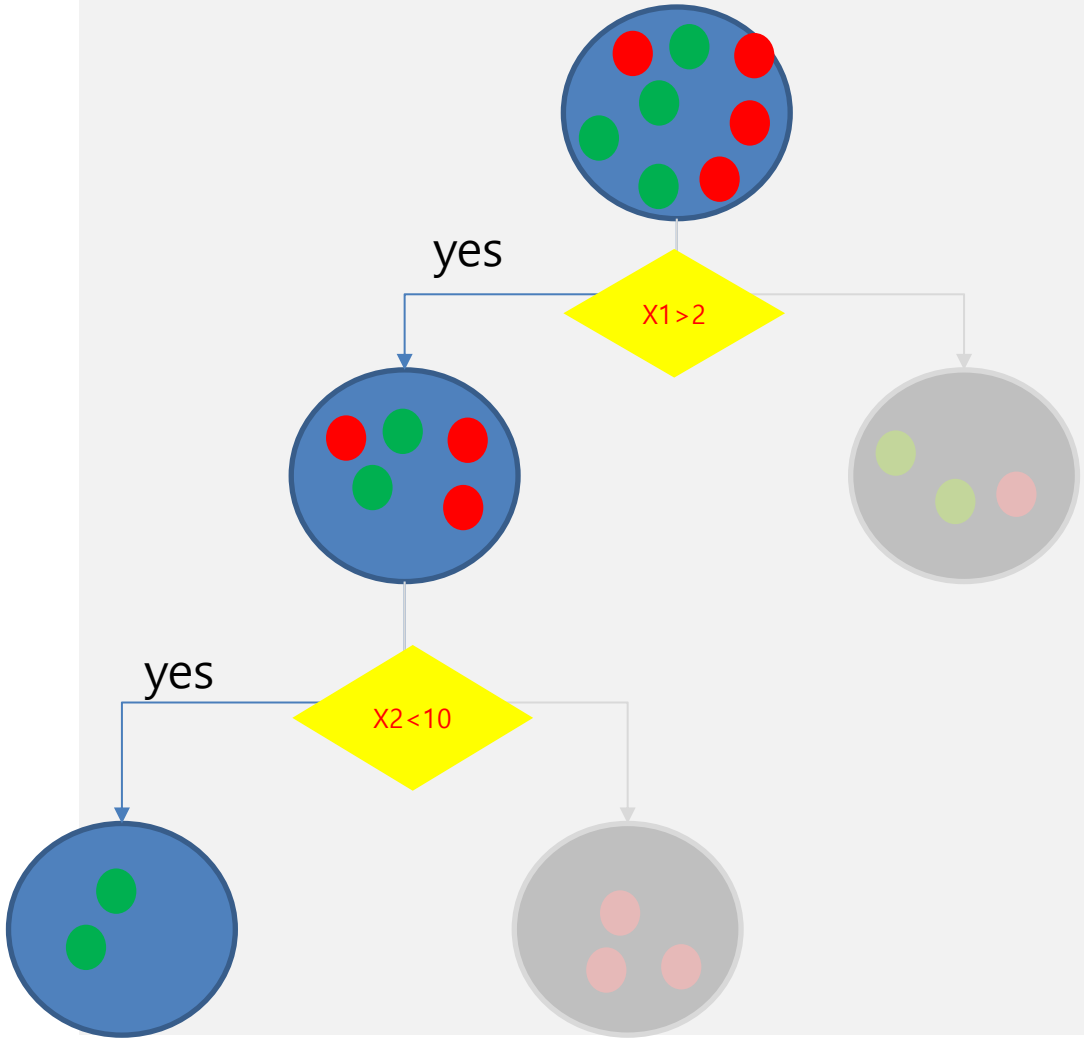
"X1이 2보다 크고
X2가 10보다 작으면
● 으로 분류됨!"



- Rule을 통한 Split의 이해
- 여러 Rule들을 통한 변수들의 interaction 이해
- Decision Tree의 출력!

1. Classification Review

Decision Tree의 이해



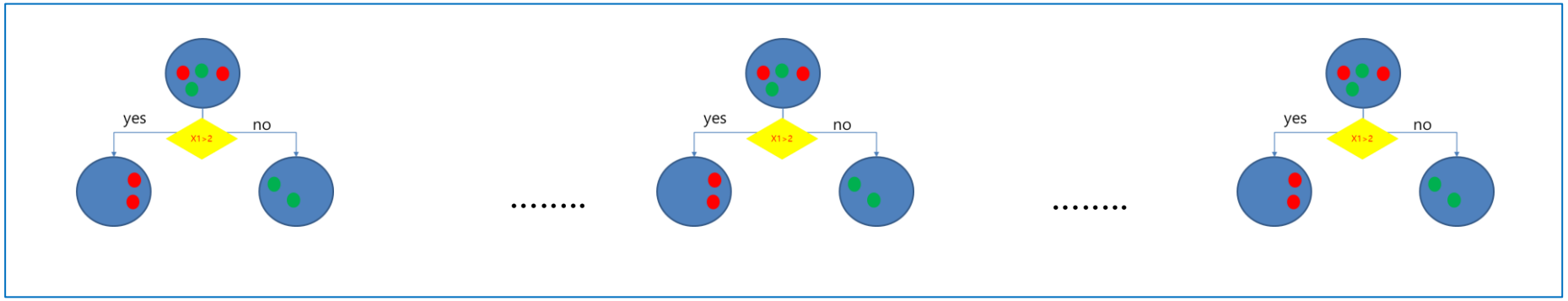
- Root
  - X1>2
    - X2< 10
      - ● 으로 분류
    - X2 >=10
      - ● 으로 분류
  - X1<=2
    - ……

2. Ensemble Model

- Ensemble 기법: 여러 분류 모형의 결과를 결합하는 기법
- Random Forest: 앙상블 학습 방법의 일종으로, 훈련 과정에서 구성한 다수의 Decision Tree로부터 Voting을 통해 결과 예측
- Bagging: 주어진 데이터에서 랜덤하게 여러 개의 같은 크기의 부분집합을 생성
- Out of Bag과 Voting: Out of Bag(OOB)는 Bagging에서 제외되는 데이터들을 의미하며, Voting은 Random Forest내 여러 Decision Tree의 결과 중 다수의 결과를 선택하는 방법

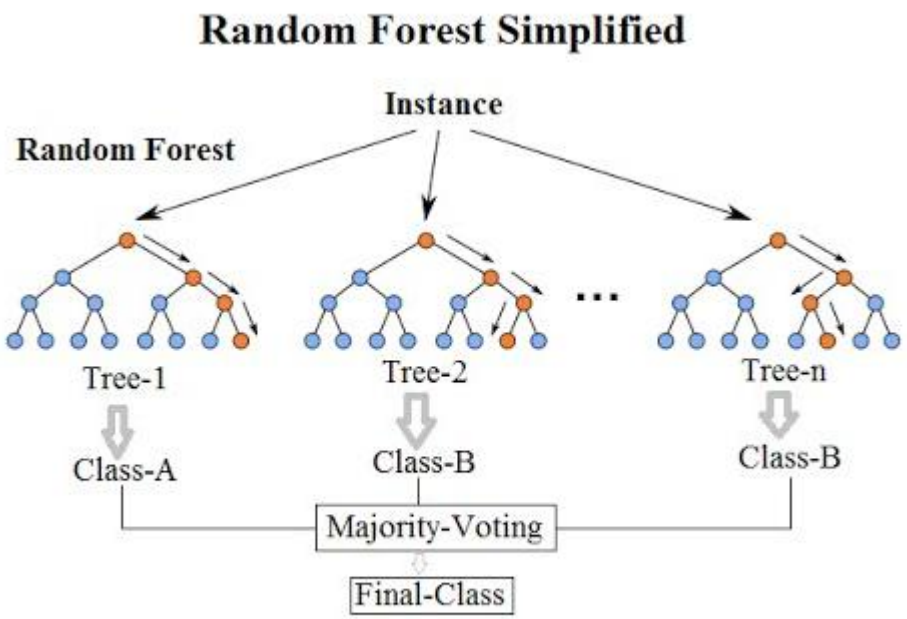
Random Forest

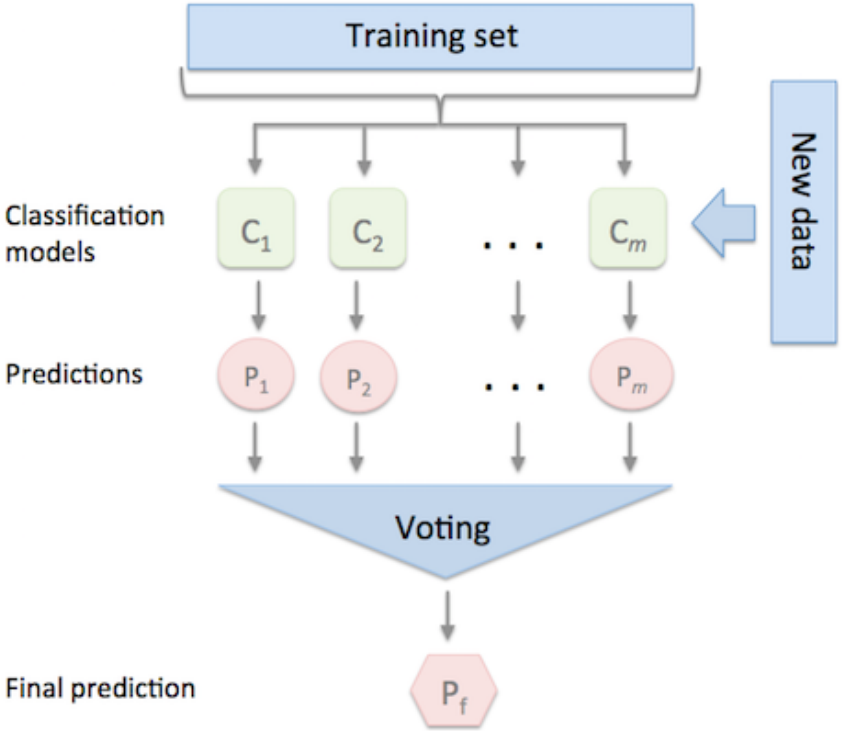
- Breiman의 " bagging " 과 변수 랜덤 선택 아이디어 기반
- 처음에는 random decision forests로 시작하여 발전
- 데이터의 다양한 경우를 반영할 수 있도록 보완
- 다양한 경우에 대한 Decision Tree를 통해 성능과 안정성을 제고



2. Ensemble Model

- **Random forest (or random forests)**
  - Ensemble classifier that consists of many decision trees and outputs the class that is the m ode of the class's output by individual trees

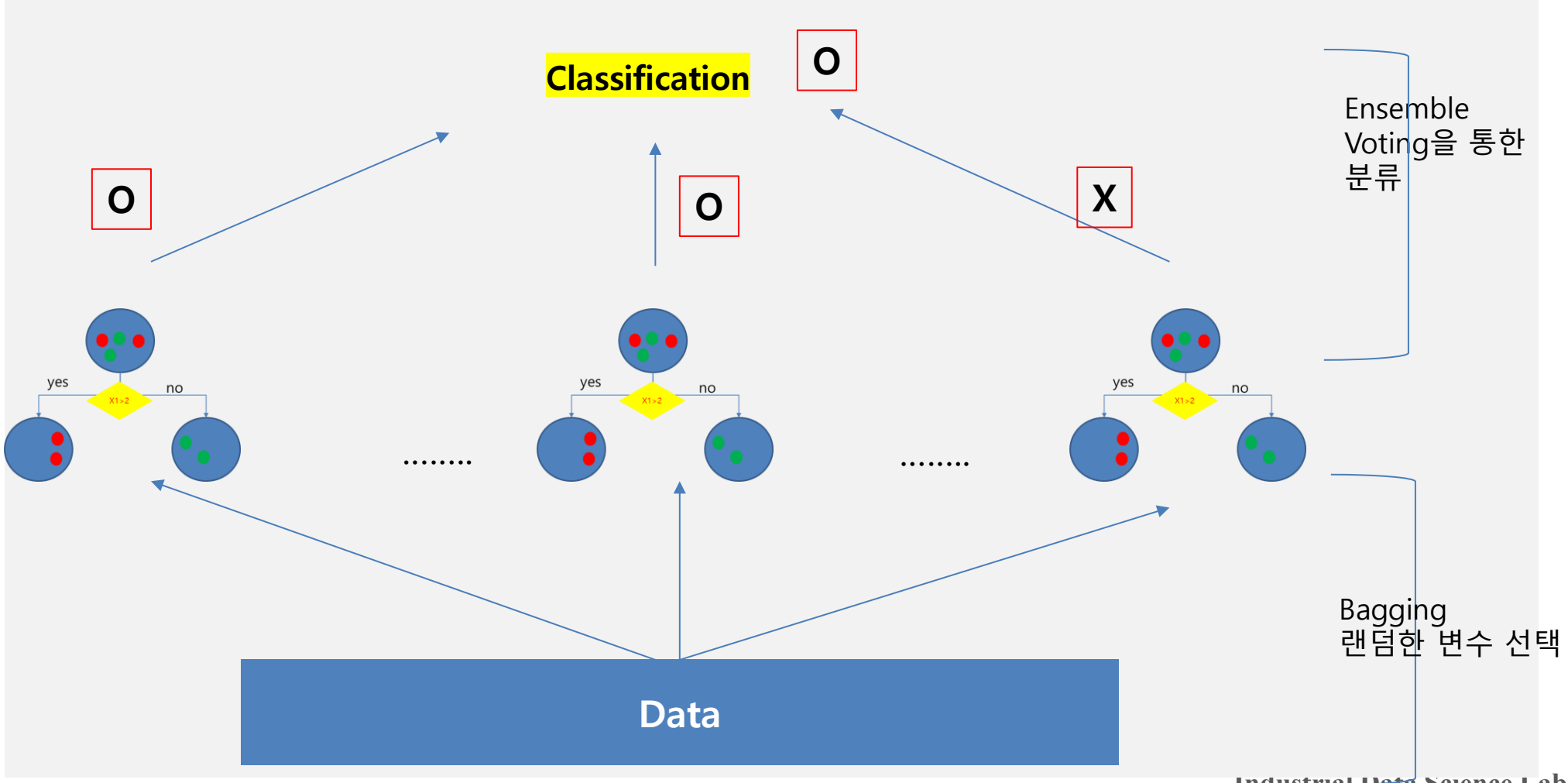




2. Ensemble Model

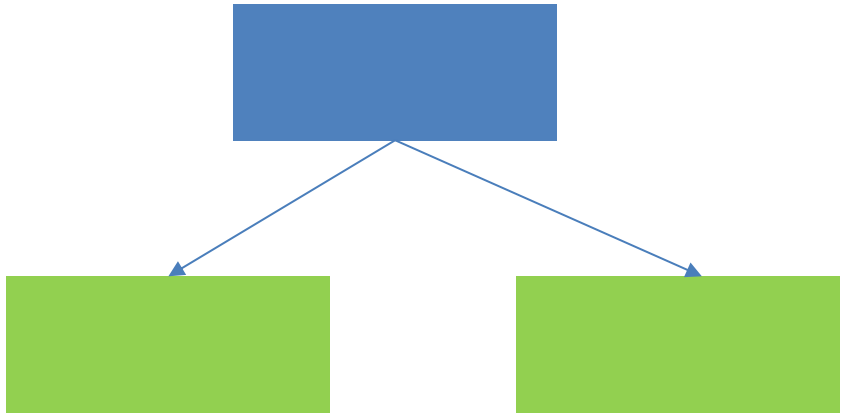
Random Forest

- 데이터의 다양한 경우를 반영할 수 있도록 보완
- 안정성을 제고



2. Ensemble Model: Adaboost

- Adaboost는 Ensemble 기법의 Boosting을 DT에 적용
- Stump로 부터 학습을 시작
  - Stump: 단순한 형태의 Tree, Weak learner

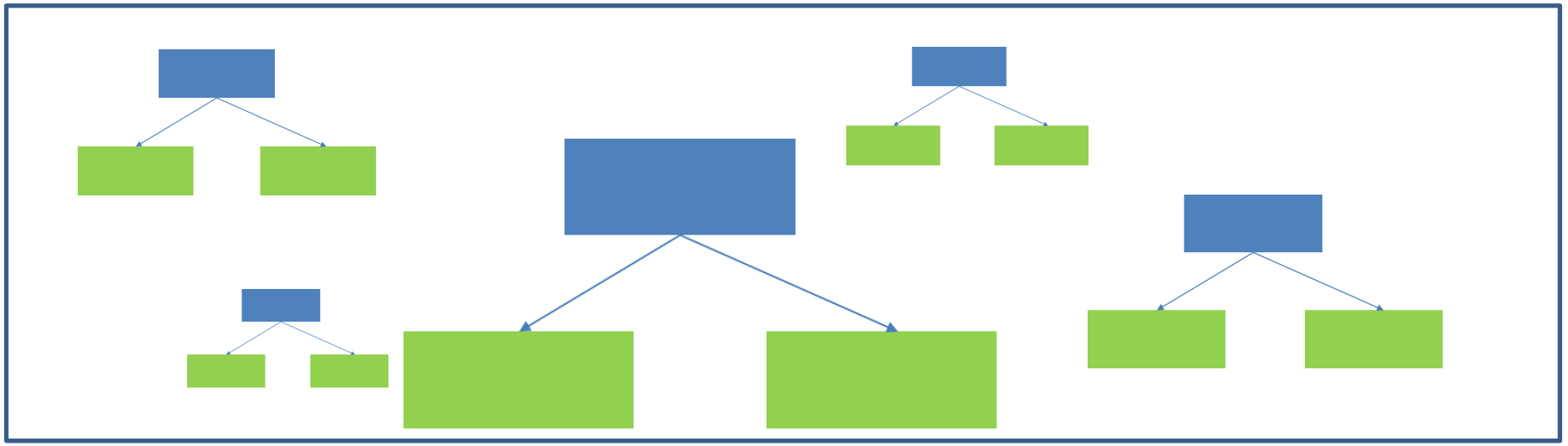


2. Ensemble Model: Adaboost

- Forest of stumps를 활용
  - Random Forest: 모든 tree는 같은 weight을 갖음
  - Adaboost: Stump마다 중요도의 차이가 존재

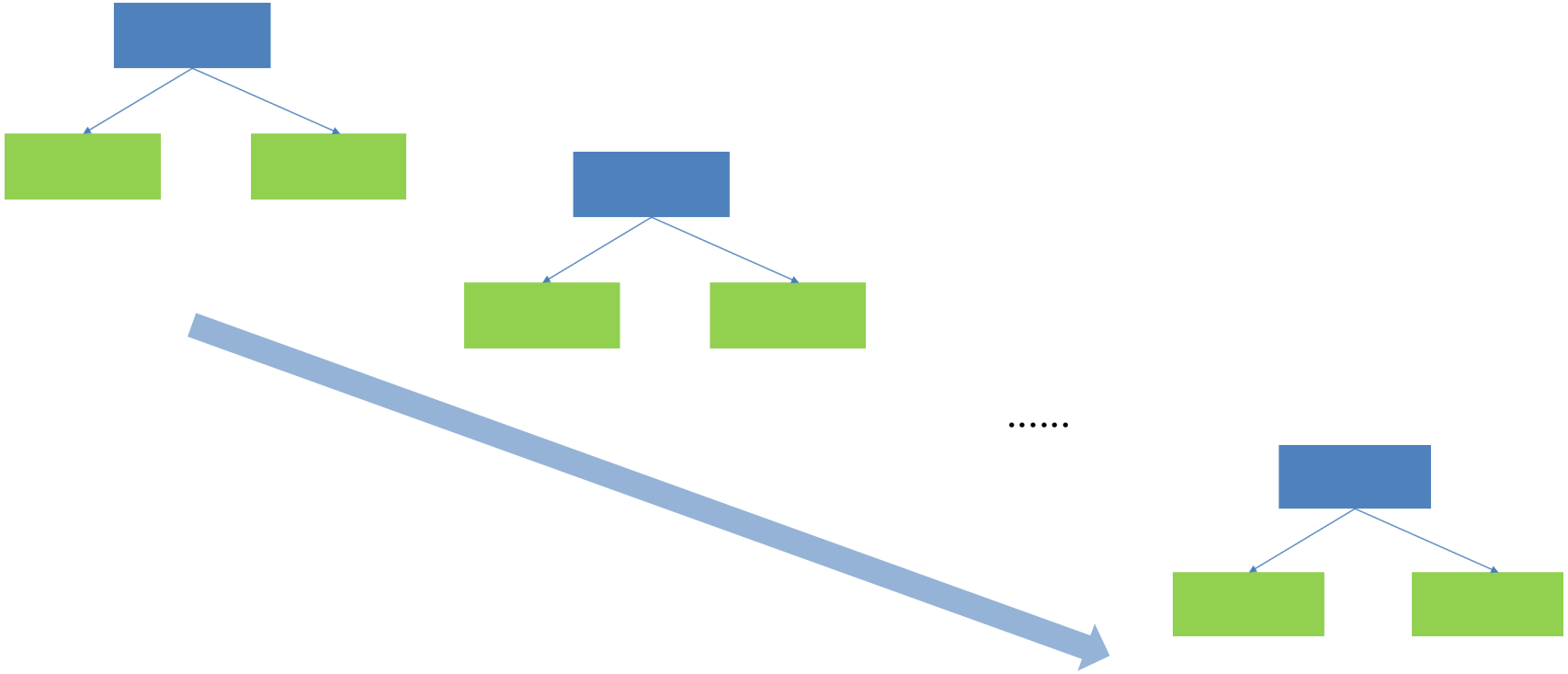
- Random Forest에서는 Tree가 같은 중요도를 지님

- Adaboost에서의 stump의 중요도: Amount of say로 표현, 클 수록 결과에 큰 영향을 미침



2. Ensemble Model: Adaboost

- Forest of stumps
  - 첫 stump는 다음 stump에 영향, 순차적으로 다음 stump에 영향을 주는 방식



2. Ensemble Model: Adaboost

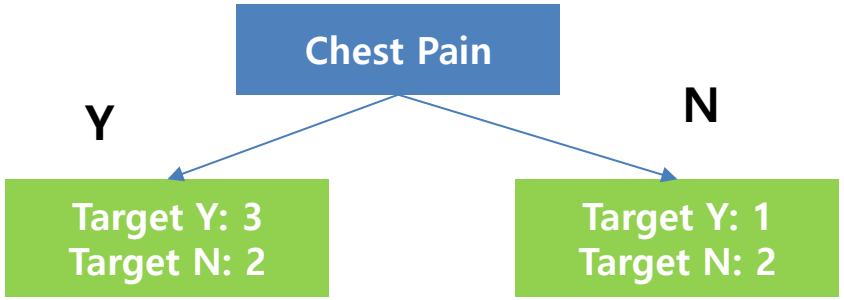
- **Example**

Target Sample Weight의 합은 1

Chest Pain	Blocked Arteries	Patient Weight	Heart Disease	Sample Weight
Y	Y	205	Y	1/8
N	Y	180	Y	1/8
Y	N	210	Y	1/8
Y	Y	167	Y	1/8
N	Y	156	N	1/8
N	Y	125	N	1/8
Y	N	168	N	1/8
Y	Y	172	N	1/8

2. Ensemble Model: Adaboost

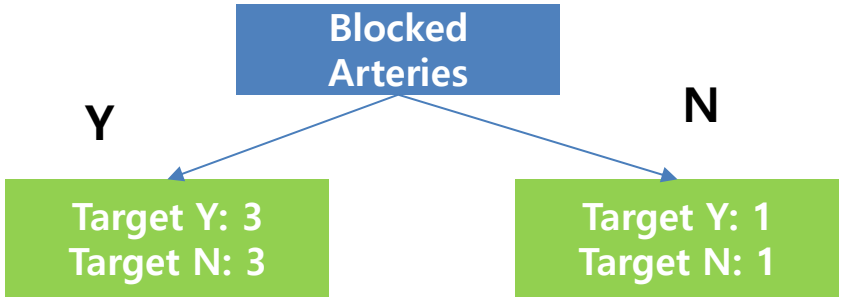
- 각 변수별 Target과의 관계



Chest Pain	Blocked Arteries	Patient Weight	Heart Disease	Sample Weight
Y	Y	205	Y	1/8
N	Y	180	Y	1/8
Y	N	210	Y	1/8
Y	Y	167	Y	1/8
N	Y	156	N	1/8
N	Y	125	N	1/8
Y	N	168	N	1/8
Y	Y	172	N	1/8

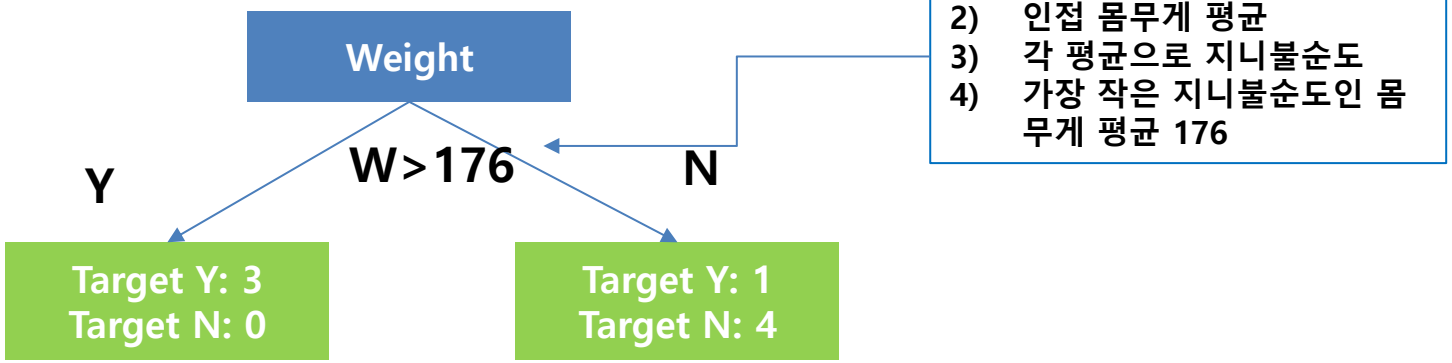
2. Ensemble Model: Adaboost

- 각 변수별 Target과의 관계



Chest Pain	Blocked Arteries	Patient Weight	Heart Disease	Sample Weight
Y	Y	205	Y	1/8
N	Y	180	Y	1/8
Y	N	210	Y	1/8
Y	Y	167	Y	1/8
N	Y	156	N	1/8
N	Y	125	N	1/8
Y	N	168	N	1/8
Y	Y	172	N	1/8

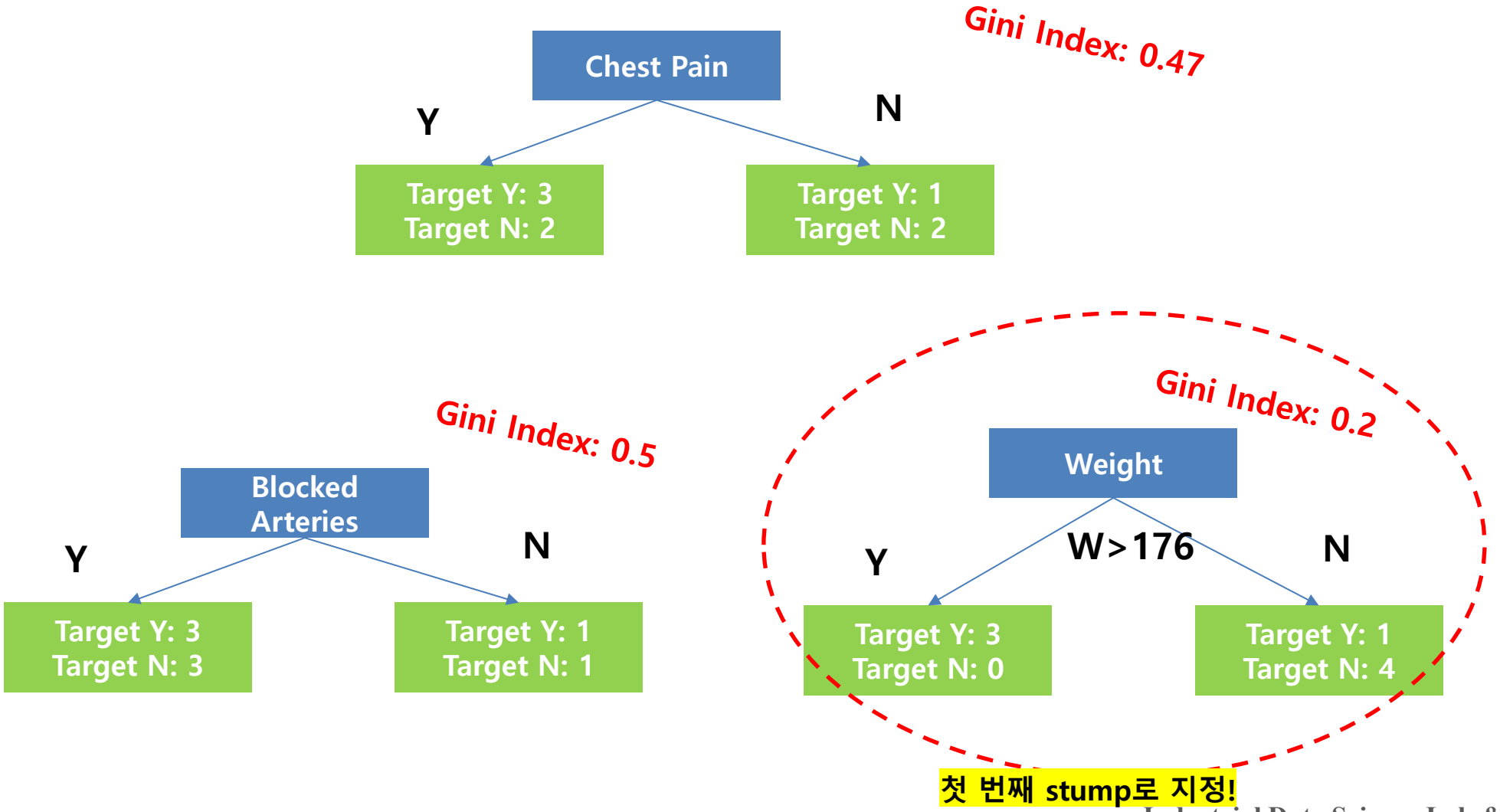
- 각 변수별 Target과의 관계



Chest Pain	Blocked Arteries	Patient Weight	Heart Disease	Sample Weight
Y	Y	205	Y	1/8
N	Y	180	Y	1/8
Y	N	210	Y	1/8
Y	Y	167	Y	1/8
N	Y	156	N	1/8
N	Y	125	N	1/8
Y	N	168	N	1/8
Y	Y	172	N	1/8

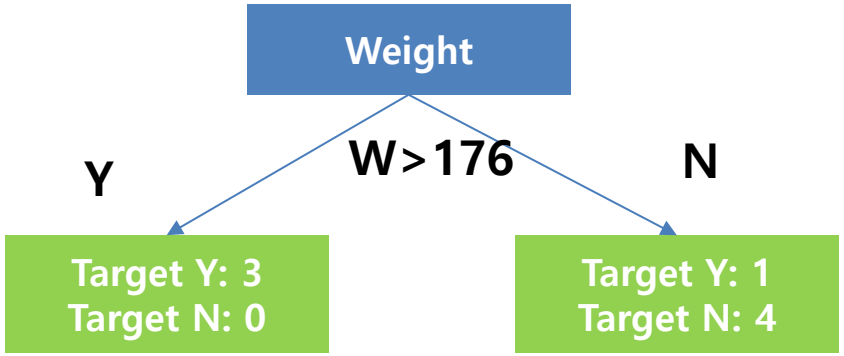
2. Ensemble Model: Adaboost

- 각 stump별 지니 계수



2. Ensemble Model: Adaboost

- 첫 stump의 Total Error



Chest Pain	Blocked Arteries	Patient Weight	Heart Disease	Sample Weight
Y	Y	205	Y	1/8
N	Y	180	Y	1/8
Y	N	210	Y	1/8
Y	Y	167	Y	1/8
N	Y	156	N	1/8
N	Y	125	N	1/8
Y	N	168	N	1/8
Y	Y	172	N	1/8

첫 stump에서 틀린 Case

Total Error = 1/8
(0~1사이 값)

Amount of Say 계산

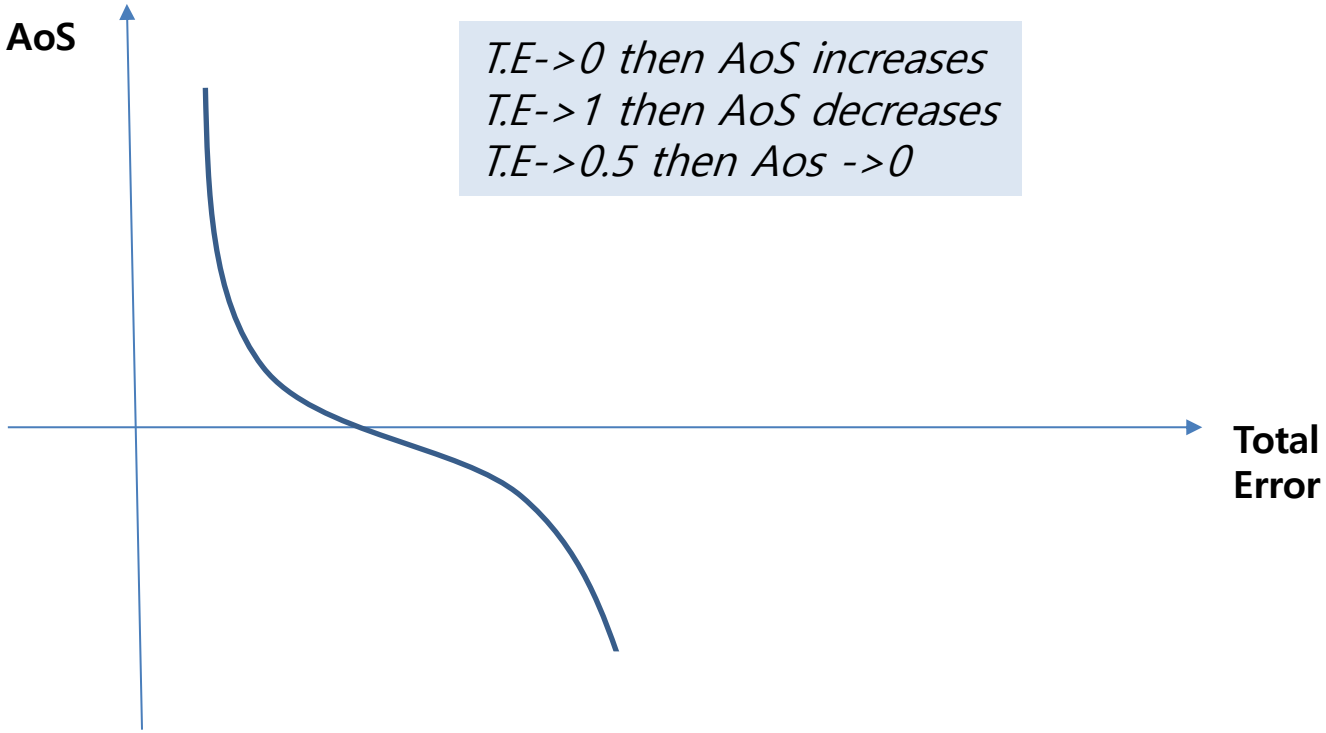
Amount of Say =
$$\frac{1}{2}log(\frac{1-Total\ Error}{Total\ Error})$$

Amount of Say = 0.97

2. Ensemble Model: Adaboost

- 첫 stump의 Total Error

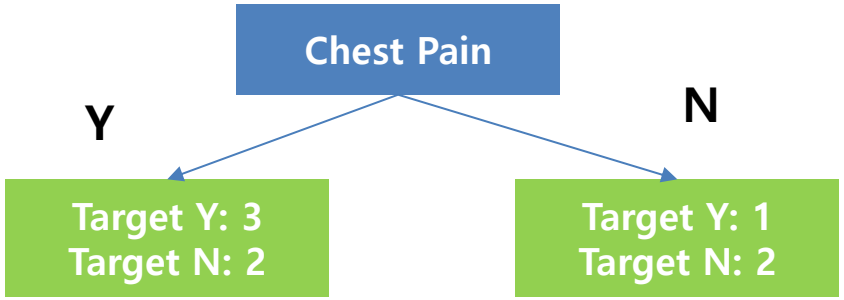
$$\text{Amount of Say} = \frac{1}{2} log\left(\frac{1-Total\ Error}{Total\ Error}\right)$$



T.E->0 then AoS increases
T.E->1 then AoS decreases
T.E->0.5 then Aos ->0

2. Ensemble Model: Adaboost

- AoS 계산 예: Chest Pain (실제 stump는 아님)



Chest Pain	Blocked Arteries	Patient Weight	Heart Disease	Sample Weight
Y	Y	205	Y	1/8
N	Y	180	Y	1/8
Y	N	210	Y	1/8
Y	Y	167	Y	1/8
N	Y	156	N	1/8
N	Y	125	N	1/8
Y	N	168	N	1/8
Y	Y	172	N	1/8

Amount of Say = 0.42

Amount of Say = $\frac{1}{2} log(\frac{1 - Total\ Error}{Total\ Error})$

Amount of Say 계산

Total Error = 3/8
(0~1사이 값)

Chest Pain stump에서 틀린 Case

- **두 번째 Stump 계산**
  - 첫 Stump가 잘못 분류한 Sample의 Weight를 높여줌
    - 이후 원래 데이터에서 샘플링을 통해 새롭게 데이터 구성
    - Weight를 활용한 샘플링
  - 첫 Stump에서 오분류된 Sample이 높은 Weight으로 더 많이 Sampling됨
    - 다음 Stump에서 이전 단계에 오분류된 Obs.에 집중

$$\boldsymbol{New\ Sample\ Weight = Sample\ Weight\ \times e^{amount\ of\ say}}$$

AoS가 크면 Weight도 증가

- 첫 stump에서의 4번째 행

Chest Pain	Blocked Arteries	Patient Weight	Heart Disease	Sample Weight
Y	Y	205	Y	1/8
N	Y	180	Y	1/8
Y	N	210	Y	1/8
Y	Y	167	Y	1/8
N	Y	156	N	1/8
N	Y	125	N	1/8
Y	N	168	N	1/8
Y	Y	172	N	1/8

$$\boldsymbol{New\ Sample\ Weight =}$$
$$\boldsymbol{\frac{1}{8} \times e^{0.97} = 0.125\ \times 2.64 = 0.33}$$

- **두 번째 Stump 계산**
  - 첫 Stump에서 정분류된 Sample은 낮은 Weight으로 덜 Sampling
    - 다음 Stump에서 이전 단계에 정분류된 Obs.는 덜 고려함

$$\boldsymbol{New\ Sample\ Weight = Sample\ Weight\ \times e^{-amount\ of\ say}}$$

AoS가 크면 Weight는 감소

Chest Pain	Blocked Arteries	Patient Weight	Heart Disease	Sample Weight
Y	Y	205	Y	1/8
N	Y	180	Y	1/8
Y	N	210	Y	1/8
Y	Y	167	Y	1/8
N	Y	156	N	1/8
N	Y	125	N	1/8
Y	N	168	N	1/8
Y	Y	172	N	1/8

$$\boldsymbol{New\ Sample\ Weight = \frac{1}{8} \times e^{-0.97} = 0.125\ \times 0.38 = 0.05}$$

- **Weight의 합이 1이 되도록 정규화**

2. Ensemble Model: Adaboost

- 두 번째 Stump를 위한 Sampling

Chest Pain	Blocked Arteries	Patient Weight	Heart Disease	Sample Weight	New Weight	Sampling 을 위한 값
Y	Y	205	Y	1/8	0.07	0~0.07
N	Y	180	Y	1/8	0.07	0.07~0.14
Y	N	210	Y	1/8	0.07	0.14~0.21
Y	Y	167	Y	1/8	0.49	0.21~0.7
N	Y	156	N	1/8	0.07	0.7~0.77
N	Y	125	N	1/8	0.07	0.77~0.84
Y	N	168	N	1/8	0.07	0.84~0.91
Y	Y	172	N	1/8	0.07	0.91~1

- 0~1사이 난수 생성
- 해당 난수 값이 속하는 행을 선택
- 중복해서 선택 가능
- Weight가 높은 행이 Sampling될 확률이 높음

2. Ensemble Model: Adaboost

- **다음 Stump를 위한 Sampling 및 학습 반복**

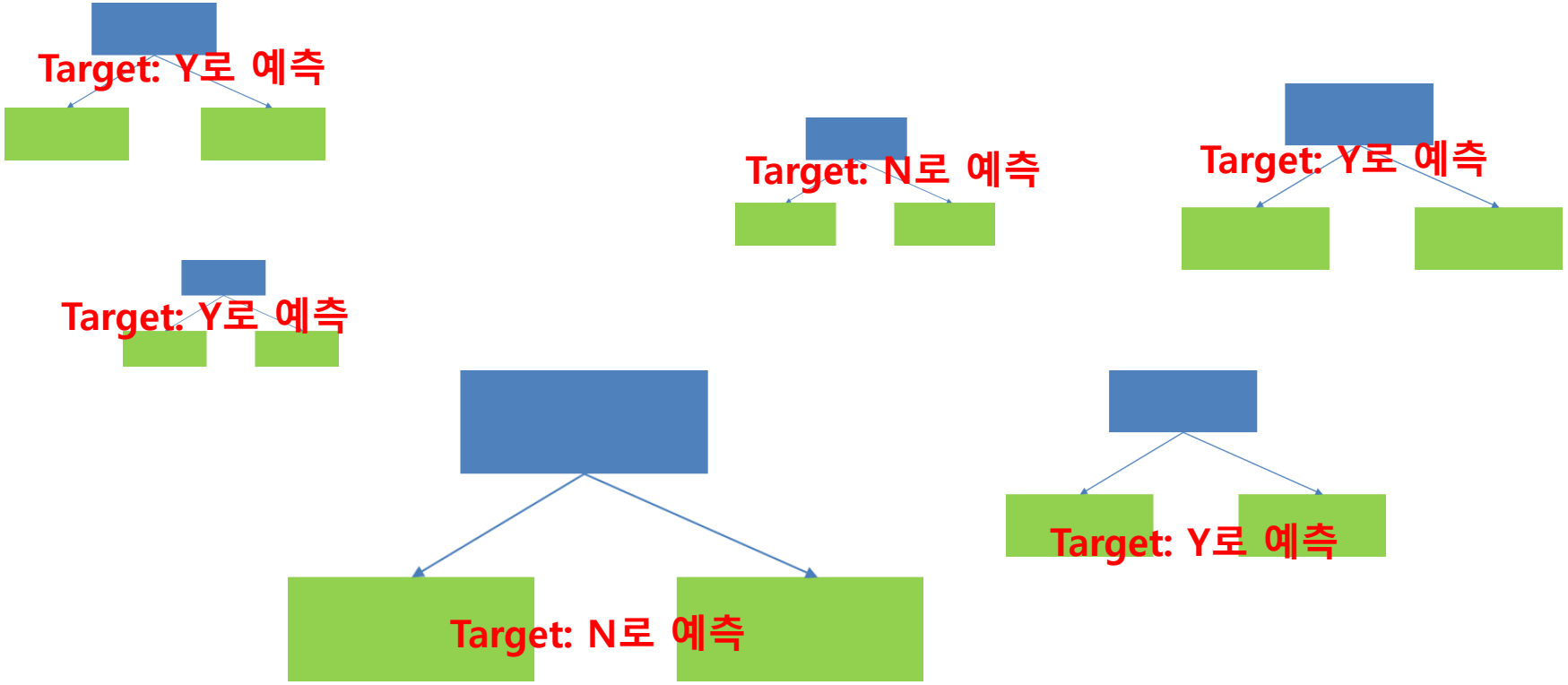
Chest Pain	Blocked Arteries	Patient Weight	Heart Disease	Sample Weight	New Weight	Sampling 을 위한 값
Y	Y	205	Y	1/8	0.07	0~0.07
N	Y	180	Y	1/8	0.07	0.07~0.14
Y	N	210	Y	1/8	0.07	0.14~0.21
Y	Y	167	Y	1/8	0.49	0.21~0.7
N	Y	156	N	1/8	0.07	0.7~0.77
N	Y	125	N	1/8	[illegible]	[illegible]
Y	N	168	N	1/8	[illegible]	[illegible]
Y	Y	172	N	1/8	[illegible]	[illegible]

- 다음 계산을 위해 가중치 초기화

Chest Pain	Blocked Arteries	Patient Weight	Heart Disease	Sample Weight
N	Y	156	N	1/8
Y	Y	167	Y	1/8
N	Y	125	N	1/8
Y	Y	167	Y	1/8
Y	Y	167	Y	1/8
Y	Y	172	N	1/8
Y	Y	205	Y	1/8
Y	Y	167	Y	1/8

2. Ensemble Model: Adaboost

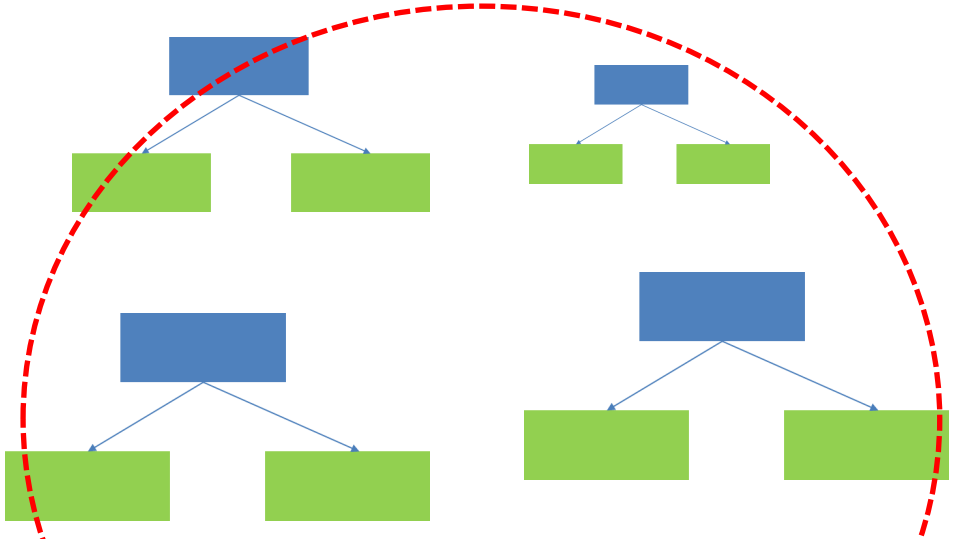
- **Adaboost의 예측 방식**
  - 주어진 X값에 대해 Forest of Stump 내의 Stump에 적용



2. Ensemble Model: Adaboost

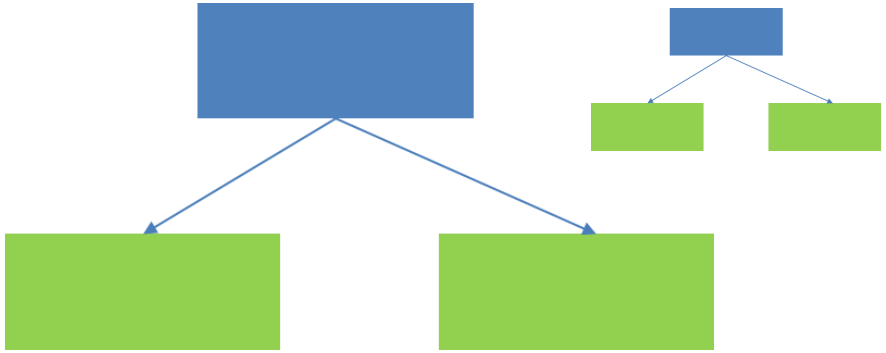
- **Adaboost의 예측 방식**
  - 주어진 X값에 대해 Forest of Stump 내의 Stump에 적용

Target: Y로 예측



- Amount of Say: 0.97, 0.32, 0.78, 0.63
- Total AoS: 2.7

Target: N로 예측



- Amount of Say: 0.41, 0.82
- Total AoS: 1.23

3. Ensemble Model: Gradient Boost

- **Adaboost VS Gradient Boost**
  - Adaboost: 여러 Stump의 순차적 계산
  - GB: leaf로 부터 시작
    - Leaf: Target에 대한 초기 추정값(예: 평균, log(odds ratio) 등)
    - Stump가 아닌 Tree를 생성: 각 tree는 leaf가 8~32개 크기 수준으로 생성

X1	X2	X3	Target
Y	12	Blue	O
Y	87	Green	O
N	44	Blue	X
Y	19	Red	X
N	32	Green	O
N	14	Blue	O

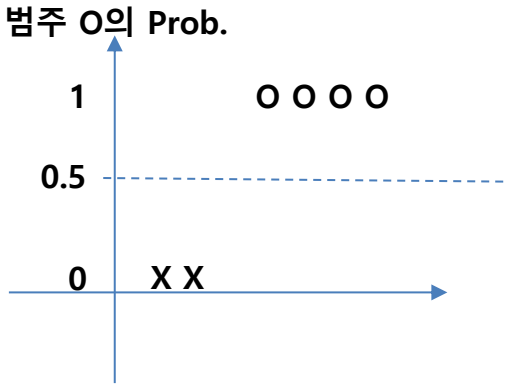
3. Ensemble Model: Gradient Boost

- **Gradient Boost for Classification, Step 1**
  - Leaf의 계산
    - X 범주 2개 대비 O범주는 4개, Odds = 4/2, leaf는 log(odds) = 0.7

X1	X2	X3	Target
Y	12	Blue	O
Y	87	Green	O
N	44	Blue	X
Y	19	Red	X
N	32	Green	O
N	14	Blue	O

- **Gradient Boost for Classification, Step 1**
  - Leaf의 계산: X 범주 2개 대비 O범주는 4개, Odds = 4/2, leaf는 log(odds) = 0.7
  - Leaf를 통한 O 범주의 확률?
    - Exponential(log(odds)) / (1+exponential(log(odds))) = 0.7
    - 이 값이 기준인 0.5와 비교하여 O, X 분류
  - Residual을 계산: 예를 들어 O는 확률 1이고, leaf 는 0.7이어서 Residual은 0.3

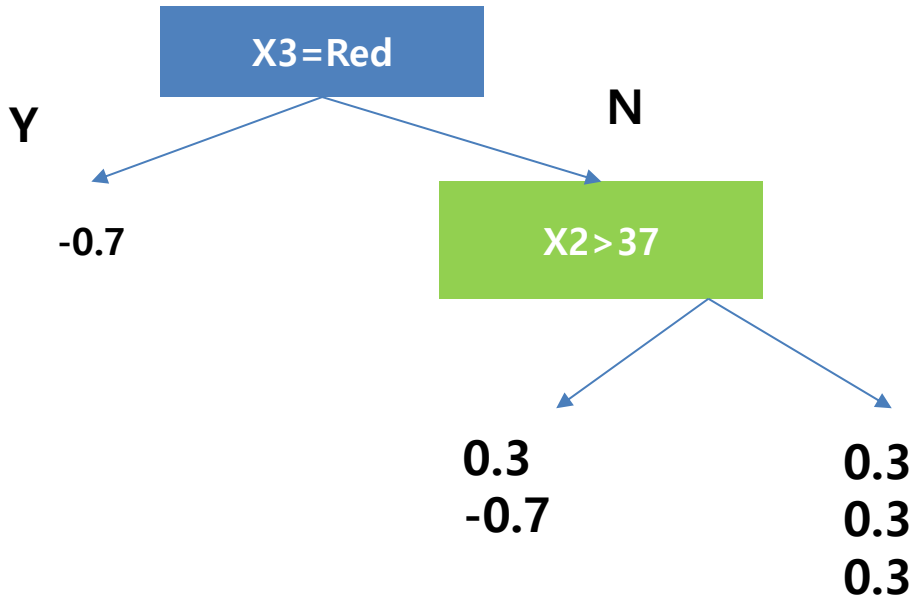
같은 X변수들로
Residual에 대한 Tree



X1	X2	X3	Target	Residual
Y	12	Blue	O	0.3
Y	87	Green	O	0.3
N	44	Blue	X	-0.7
Y	19	Red	X	-0.7
N	32	Green	O	0.3
N	14	Blue	O	0.3

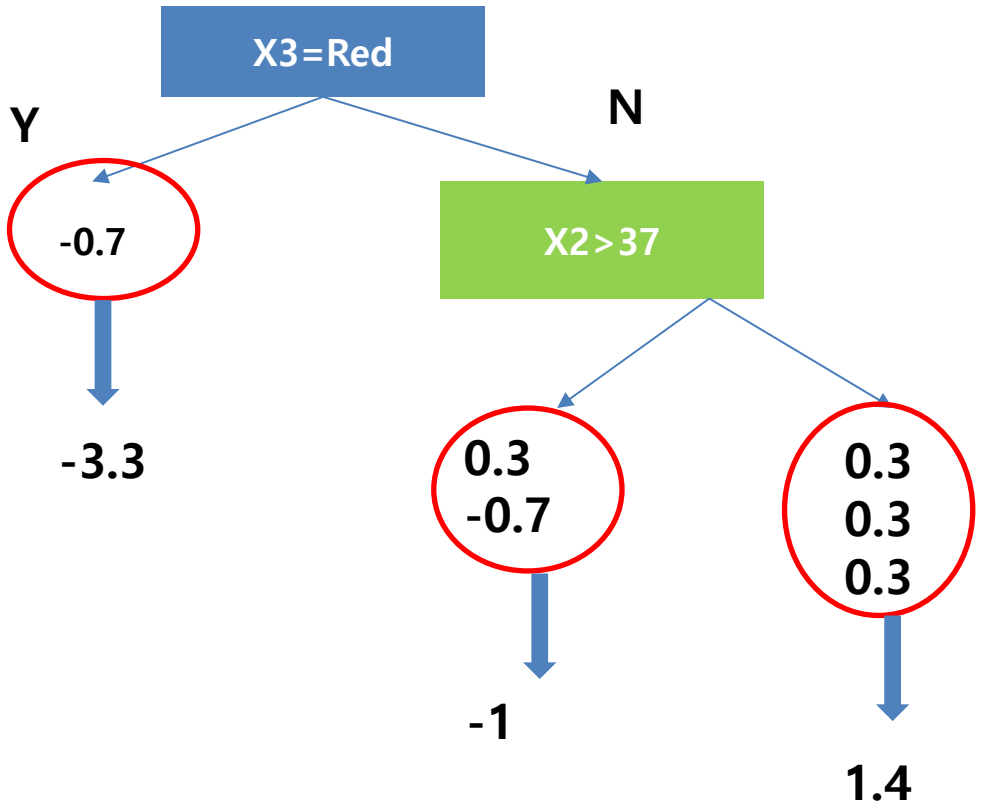
3. Ensemble Model: Gradient Boost

- **Gradient Boost, Step 1**
  - **1st Tree**
    - leaf의 수를 8~32로 제한하며 그 범위내에서 tree 생성



3. Ensemble Model: Gradient Boost

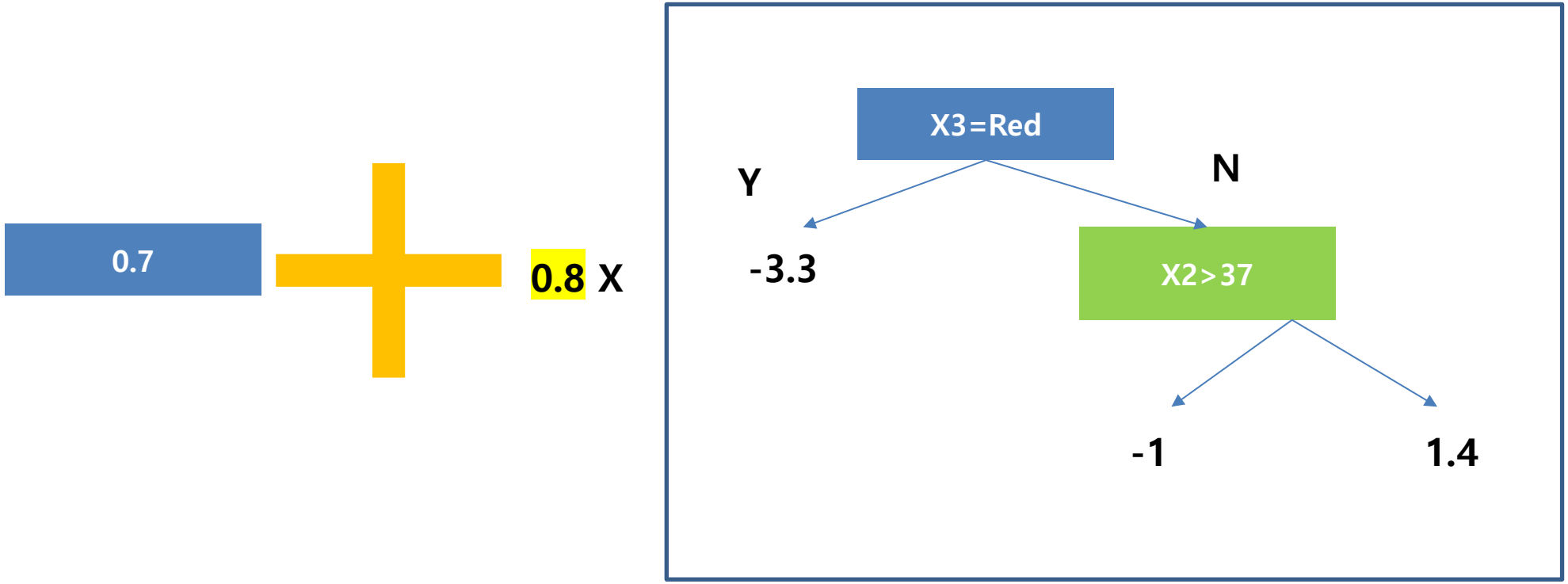
- **Gradient Boost, Step 1**
  - **1st Tree**



$$\frac{\sum Residuals}{\sum(Previous\ Prob\ \times (1 - Previous\ Prob))}$$

3. Ensemble Model: Gradient Boost

- **Gradient Boost, Step 2**
  - **Leaf 의 initial prediction에 tree에 학습율 반영하여 계산**
  - **Leaf + 1st Tree**



3. Ensemble Model: Gradient Boost

- **Gradient Boost for Classification, Step 3**
  - 각 범주에 대한 발생 확률 계산
    - 1st Obs의 업데이트된 log(odds)는 1.8
      - Leaf 0.7 + 1.4(from tree) X 0.8 = 1.8
      - 1st Obs의 확률: $\frac{e^{1.8}}{1+e^{1.8}}$

X1	X2	X3	Target	Residual	Prob.
Y	12	Blue	O	0.3	0.9
Y	87	Green	O	0.3	0.5
N	44	Blue	X	-0.7	0.5
Y	19	Red	X	-0.7	0.1
N	32	Green	O	0.3	0.9
N	14	Blue	O	0.3	0.9

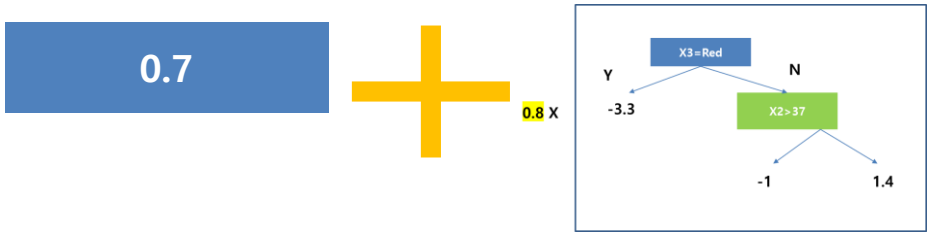
3. Ensemble Model: Gradient Boost

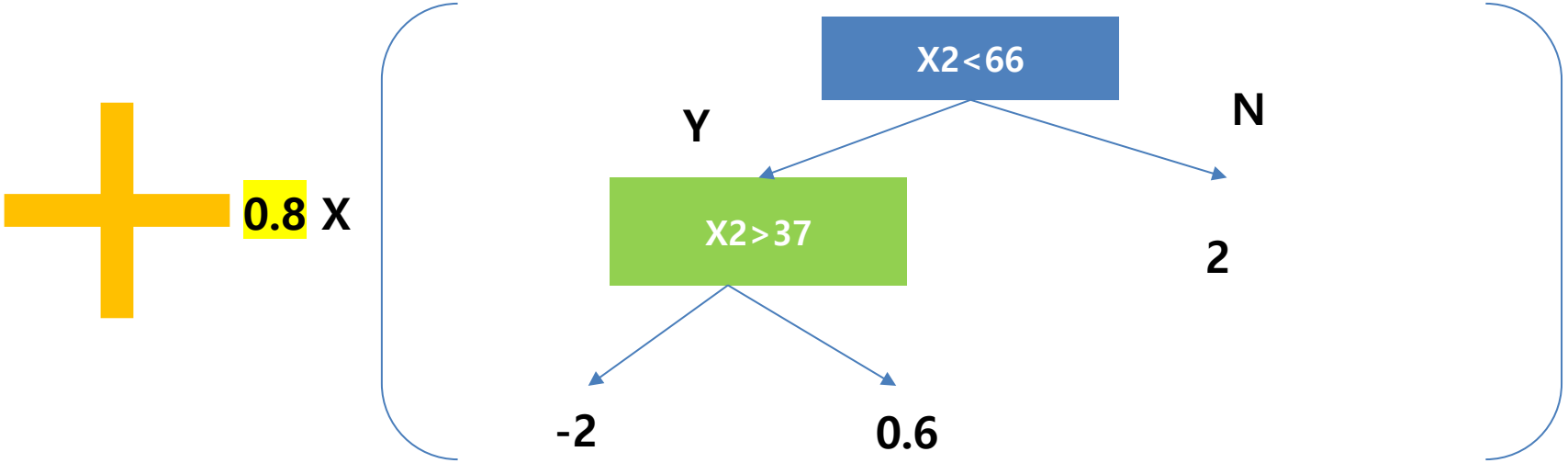
- **Gradient Boost for Classification, Step 3**
  - Residual 다시 계산, 다음 tree 생성

X1	X2	X3	Target	Residual	Prob.	New Residual
Y	12	Blue	O	0.3	0.9	1-0.9
Y	87	Green	O	0.3	0.5	1-0.5
N	44	Blue	X	-0.7	0.5	0-0.5
Y	19	Red	X	-0.7	0.1	0-0.1
N	32	Green	O	0.3	0.9	1-0.9
N	14	Blue	O	0.3	0.9	1-0.9

3. Ensemble Model: Gradient Boost

- **Gradient Boost, Step 3**
  - Learning Rate: 0~1사이, 이 예에서는 0.1 사용





3. Ensemble Model: Gradient Boost

- **Gradient Boost for Regression**
  - GB: leaf로 부터 시작
    - Leaf: Target에 대한 초기 추정값(예: 평균, log(odds ratio) 등)
    - Stump가 아닌 Tree를 생성: 각 tree는 leaf가 8~32개 크기 수준으로 생성

Target

Height	Color	Gender	Weight
1.6	B	M	88
1.6	G	F	76
1.5	B	F	56
1.8	R	M	73
1.5	G	M	77
1.4	B	F	57

3. Ensemble Model: Gradient Boost

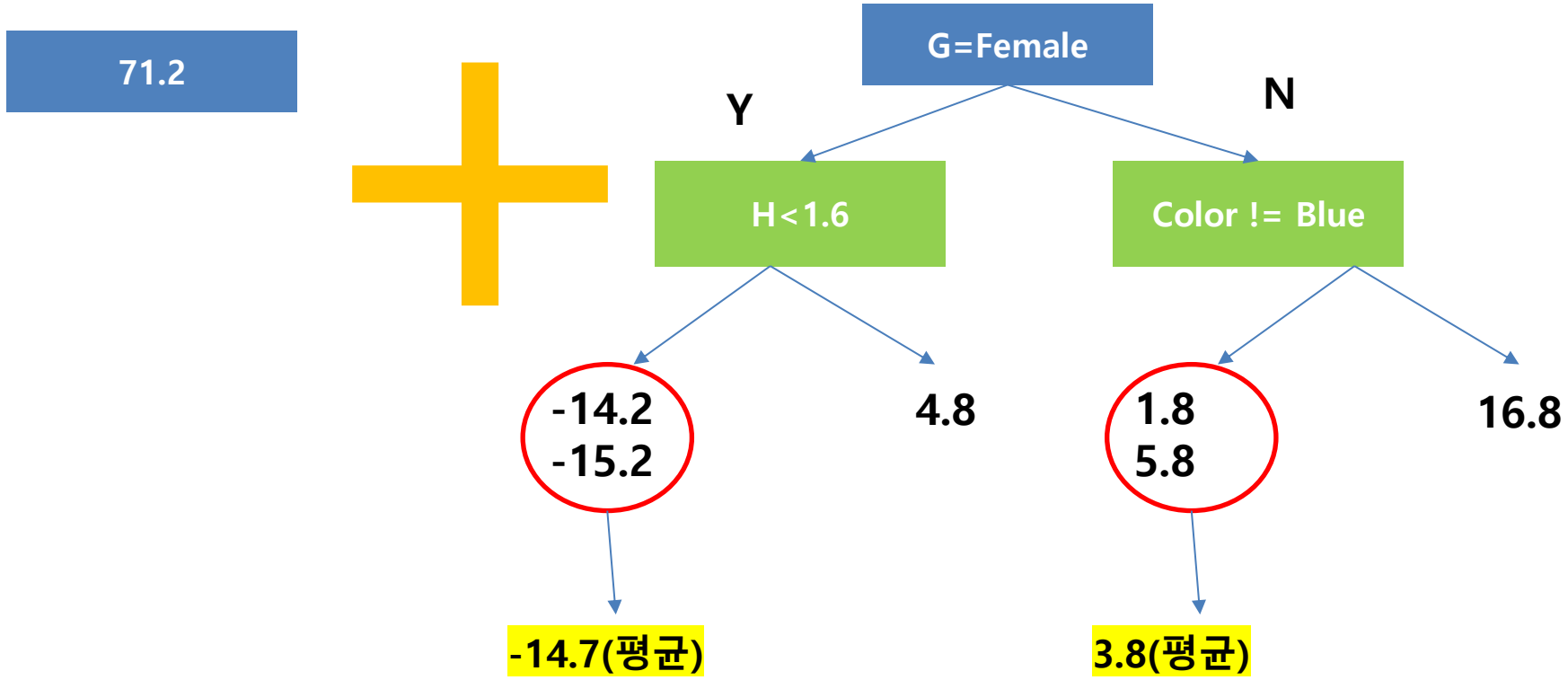
- **Gradient Boost, Step 1**
  - Leaf의 계산
  - Target인 Weight의 평균: 71.2
  - Residual을 계산: 실제값과 예측값의 차이(error)

같은 X변수들로
Residual에 대한 Tree

Height	Color	Gender	Weight	Residual
1.6	B	M	88	16.8
1.6	G	F	76	4.8
1.5	B	F	56	-15.2
1.8	R	M	73	1.8
1.5	G	M	77	5.8
1.4	B	F	57	-14.2

3. Ensemble Model: Gradient Boost

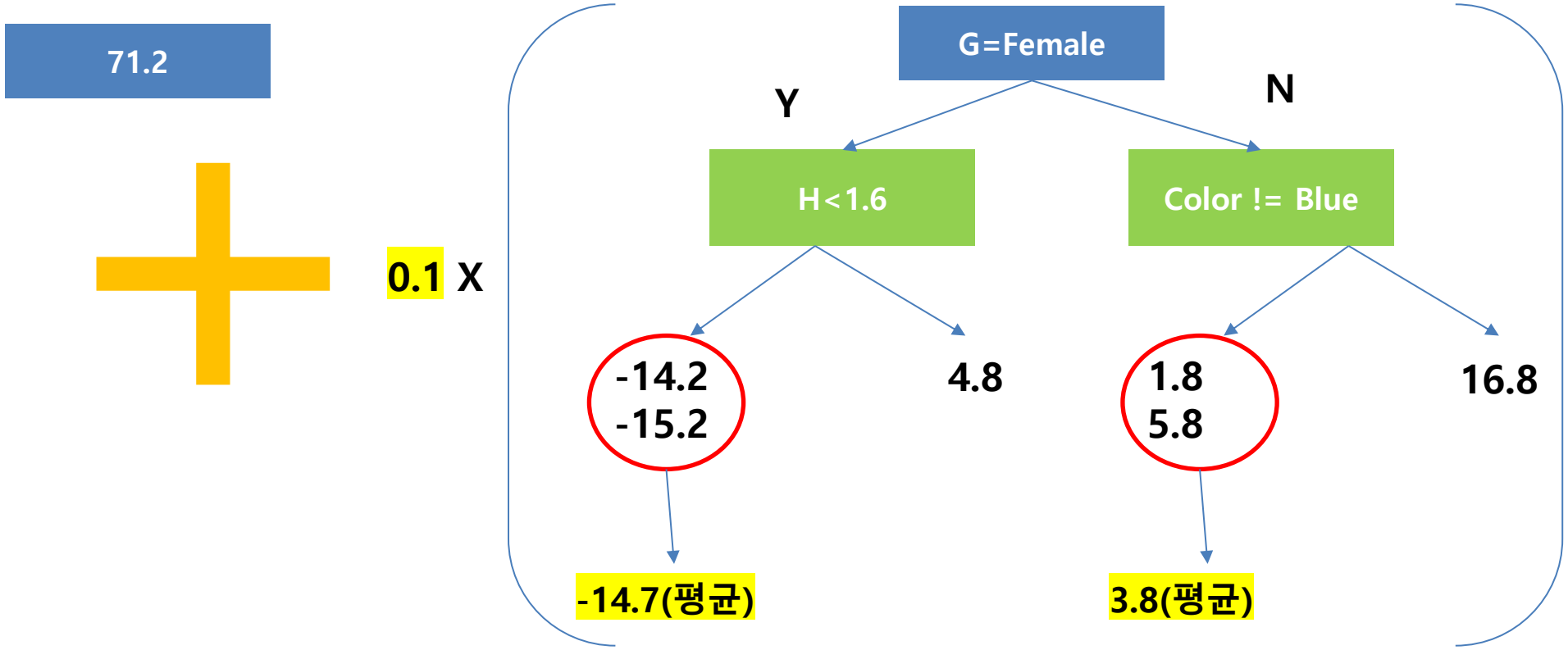
- **Gradient Boost, Step 1**
  - **Leaf + 1st Tree**



- **Male, Blue인 경우 예측 예시:**
  - 71.2 + 16.8 = 88 (관측치와 동일하지만 과적합)
  - Bias는 작지만 Variance 큰 상태

3. Ensemble Model: Gradient Boost

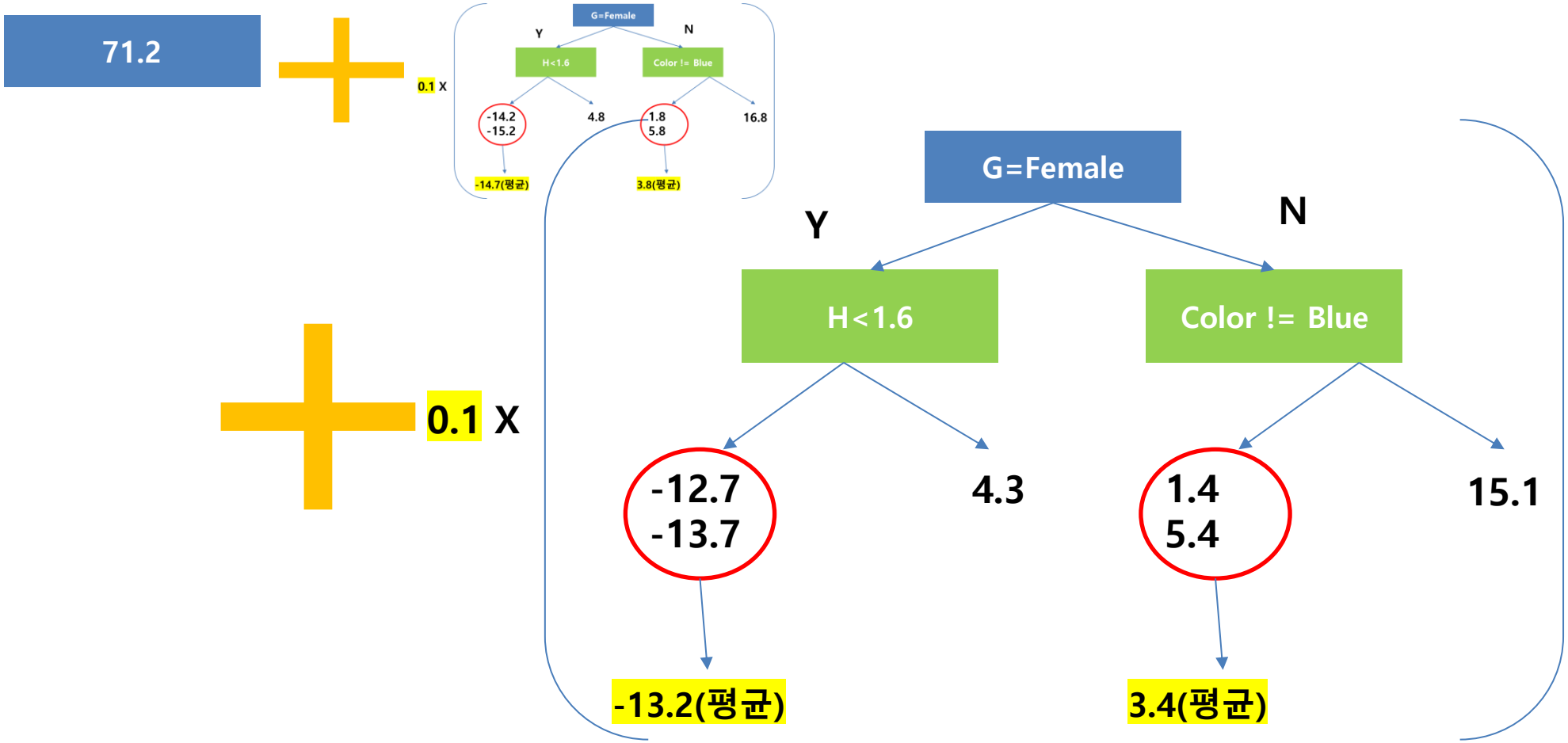
- **Gradient Boost, Step 2**
  - 과적합 방지, 학습속도 조절을 위한 학습율 도입
  - Learning Rate: 0~1사이, 이 예에서는 0.1 사용



- **Male, Blue인 경우 예측 예시:** 71.2 + 0.1 X 16.8 = 72.9
  - 실제값에 가까워지지만, 그 정도가 조절됨 (Gradient의 개념)
  - Variance를 낮게 유지할 수 있음

3. Ensemble Model: Gradient Boost

- **Gradient Boost, Step 3**
  - Learning Rate: 0~1사이, 이 예에서는 0.1 사용



- **H=1.6, Male, Blue인 경우 예측 예시:** 71.2 + 0.1 X 16.8 + 0.1 X 15.1 = 74.4

3. Ensemble Model: Gradient Boost

- **Gradient Boost, Step 3**
  - 학습율 반영 예측값을 통한 두 번째 Residual 계산

같은 X변수들로 New Residual에 대한 Tree

Height	Color	Gender	Weight	Residual	Residual(new)
1.6	B	M	88	16.8	15.1
1.6	G	F	76	4.8	4.3
1.5	B	F	56	-15.2	-13.7
1.8	R	M	73	1.8	1.4
1.5	G	M	77	5.8	5.4
1.4	B	F	57	-14.2	-12.7

Residual 크기 감소

3. Ensemble Model: Gradient Boost

- **Gradient Boost**
  - 위의 과정을 계속 반복
    - 정해진 iteration한도 까지 반복
    - 또는 이전 단계와 이후 단계의 Residual 차이가 없을 때까지 반복

  - 매 iteration에서의 Tree의 leaf는 8~32개 사이에서 생성
  - 매 iteration마다 다르게 생성
    - 1st tree: leaf 8개
    - 2nd tree: leaf 32개
    - 3rd tree: leaf 16개
    - …

4. Ensemble Model: Light GBM, XGBoost

- **Light GBM : MS Research, 2017**
  - Leaf-wise 트리 성장
    - Level-wise 대신 손실 최소 리프를 우선 분기하여 정확도 향상
  - GOSS 샘플링
    - Gradient-based One-Side Sampling: 중요 샘플 유지, 경량 샘플 랜덤 제거
  - Exclusive Feature Bundling: 희소 특성을 합쳐 차원 축소 및 속도 향상
  - 원-핫 인코딩 없이 범주형 특성 직접 처리 가능
- 장점
  - 학습 속도: vs XGBoost 대비 10~20배
  - 낮은 메모리 사용: 히스토그램 기반 최적화
  - 수백만 레코드 처리 적합
  - 소규모 데이터에서의 과적합

4. Ensemble Model: Light GBM, XGBoost

- **LightGBM**
  ① 데이터 수치형/범주형 특성 확인
  ② 각 특성의 값을 히스토그램 빈(bin)으로 이산화(discretize)
    - 연속적인 값을 정해진 구간으로 묶어 메모리와 연산 감소
  ③ 초기 예측값 $F_0(x)$ 설정: y의 평균값
  ④ Gradient 계산: 현재 예측값과 실제 Y 잔차(residual)
  ⑤ GOSS(Gradient-based One-Side Sampling) 적용
    - 그래디언트가 큰 샘플은 전부 유지하고, 작은 샘플은 일부만 랜덤 추출
  ⑥ EFB(Exclusive Feature Bundling): sparse한 변수들을 하나로 묶어 차원 축소
  ⑦ 히스토그램 기반 각 특성의 최적 분기점 탐색
  ⑧ Leaf-wise 전략으로 트리 성장
    - 모든 리프를 균등하게 분기하는 Level-wise와 달리, 손실 감소가 가장 큰 리프 하나를 우선 분기
  ⑨ $F(x) \leftarrow F(x) + \eta \cdot h(x)$: Tree $h(x)$에 학습률 η를 곱해 현재 예측값에 더하기
    ① Validation의 loss 측정 및 early stopping 조건 확인
  ⑩ 최종 모델: 모든 tree 예측값을 합산한 앙상블

4. Ensemble Model: Light GBM, XGBoost

- **LightGBM**

```
import lightgbm as lgb

params = {
    'objective': 'binary',
    'boosting_type': 'gbdt',
    'num_leaves': 31,
    'learning_rate': 0.05,
    'feature_fraction': 0.9,
    'bagging_fraction': 0.8,
    'min_data_in_leaf': 20,
    'n_estimators': 500
}

dtrain = lgb.Dataset(X_train, y_train)
model = lgb.train(
    params, dtrain,
    num_boost_round=100,
    valid_sets=[dtest]
)
pred = model.predict(X_test)
```

파라미터	설명
num_leaves	트리의 최대 리프 수 (복잡도 제어)
learning_rate	각 트리의 기여 가중치
feature_fraction	각 트리에 사용할 특성 비율
min_data_in_leaf	리프의 최소 샘플 수 (과적합 방지)
n_estimators	트리 수 (early stopping 권장)

early_stopping_rounds 설정으로 과적합 방지

4. Ensemble Model: Light GBM, XGBoost

- **XGBoost(eXtreme Gradient Boosting)**
  - Chen & Guestrin, 2016
  - 정규화(L1 & L2): 과적합 방지를 위한 정규화 항을 목적함수에 직접 포함, 안정성과 정확도가 높음
  - Pruning: max_depth 도달 이후 gain이 음수인 분기를 역방향으로 제거
  - 학습 속도는 다소 느리며, 병렬 처리 지원
  - 목적 함수 설정이 유연하여 사용자 정의 목적함수 설정 가능
  - Feature_importance를 지원하여 상대적으로 해석이 용이

3. Ensemble Model: Gradient Boost

- **XGBoost 절차**

① 입력 데이터를 DMatrix 형식으로 변환: 결측값 처리, 희소 행렬 최적화

② 초기 예측값 $F_0(x)$ 설정: y의 평균 또는 log(odds)

③ 현재 예측값 기준, 목적함수의 1차 미분(g, gradient)과 2차 미분(h, hessian) 계산: 2차 테일러 근사 사용

④ 각 특성 정렬 또는 근사 분위수(approximate quantile sketch)로 분기 후보 탐색

⑤ 각 후보 분기점에 대해 Gain 계산: $\text{Gain} = \frac{1}{2}\cdot[GL^2/(HL+\lambda) + GR^2/(HR+\lambda) - (GL+GR)^2/(HL+HR+\lambda)] - \gamma$

- γ(최소 gain)보다 작으면 분기하지 않음
- Gain이 최대인 분기점 선택, Level-wise(깊이 우선)로 Tree 형성

⑥ max_depth까지 성장, 이후 음수 Gain인 가지를 역방향으로 가지치기(pruning)

⑦ 각 leaf의 최적 가중치 w 계산: $w = -G/(H+\lambda)$, 이때 λ는 L2 정규화 계수

- L1 정규화(α) 설정 시, leaf에 추가로 shrinkage 적용

⑧ $F(x) \leftarrow F(x) + \eta\cdot h(x)$: 만들어진 Tree에 학습률 η 곱하여 예측값에 더함.

- Validation loss 측정 및 early stopping 조건 확인

⑨ 모든 트리의 가중 합산 앙상블로 최종 모형

4. Ensemble Model: Light GBM, XGBoost

- **XGBoost**

```
import xgboost as xgb

params = {
    'objective': 'binary:logistic',
    'max_depth': 6,
    'learning_rate': 0.1,
    'n_estimators': 300,
    'subsample': 0.8,
    'colsample_bytree': 0.8,
    'reg_alpha': 0.1,
    'reg_lambda': 1.0,
    'eval_metric': 'auc'
}

model = xgb.XGBClassifier(**params)
model.fit(
    X_train, y_train,
    eval_set=[(X_test, y_test)],
    early_stopping_rounds=20,
    verbose=False
)
pred = model.predict(X_test)
```

파라미터	설명
max_depth	각 트리의 최대 깊이
subsample	행 샘플링 비율
colsample_bytree	열(특성) 샘플링 비율
reg_alpha	L1 정규화 계수
reg_lambda	L2 정규화 계수

5. Ensemble Model: CatBoost

- **CatBoost(Categorical Boosting)**
  - Yandex, 2017, 범주형 데이터 특화
  - Ordered Target Encoding으로 범주형 데이터를 자동 변환 (데이터 누수 방지)
  - Ordered Boosting: 학습 순서를 랜덤화 예측 편향(Prediction Shift) 방지, 과적합 방지
  - 대칭 Tree: Oblivious Trees 사용으로 빠른 예측 속도와 과적합 방지
  - 결측값, 범주형 변수 처리를 자동으로 수행 — 전처리 부담 최소화
  - 빠른 예측 속도
  - 파라미터 민감도 낮음: 우수한 성능

5. Ensemble Model: CatBoost

- **CatBoost(Categorical Boosting)**

```
from catboost import CatBoostClassifier

# 범주형 열 인덱스 지정
cat_features = [0, 2, 5]  # 인덱스

model = CatBoostClassifier(
    iterations=500,
    learning_rate=0.05,
    depth=6,
    l2_leaf_reg=3,
    cat_features=cat_features,
    eval_metric='AUC',
    early_stopping_rounds=50,
    verbose=100
)

model.fit(
    X_train, y_train,
    eval_set=(X_test, y_test),
    plot=True  # 학습 곡선 시각화
)
pred = model.predict(X_test)
```

파라미터	설명
iterations	트리 수 (n_estimators)
depth	트리 최대 깊이 (기본 6)
l2_leaf_reg	L2 정규화 계수
cat_features	범주형 특성 인덱스 목록
border_count	수치형 특성 분기 후보 수

cat_features에 범주형 열만 지정, 원-핫 인코딩 없이 자동으로 최적 처리

5. Ensemble Model: CatBoost

- **CatBoost(Categorical Boosting)**
  ① 데이터 중 범주형 변수 인덱스 지정
  ② Ordered Boosting: 학습 데이터를 랜덤하게 섞어 순서를 무작위화
    - 범주형 특성에 Ordered Target Encoding 적용: 각 샘플의 인코딩 값 계산 시 해당 샘플 이전의 데이터만 사용하여 타깃 누수(target leakage) 방지
    - 수치형 특성: 경계값(border) 후보 생성해 이산화, border_count로 후보 수 조절
  ③ 초기 예측값 $F_0(x)$ 설정: Y의 평균
  ④ Gradient: 현재 예측값과 실제 Y의 차이, Ordered 방식으로, 각 샘플의 그래디언트를 이전 데이터로만 구성된 모델에서 계산하여 예측 편향(prediction shift)을 최소화
    - 대칭 트리(Oblivious Tree): 트리의 모든 깊이에서 동일한 분기 조건을 적용, 구조가 좌우 대칭
  ⑤ 각 depth에서 모든 변수와 분기값 후보를 탐색: 손실 감소가 가장 큰 단일 조건 선택
  ⑥ Max_depth까지 학습, 이후 leaf 가중치 계산
  ⑦ $F(x) \leftarrow F(x) + \eta \cdot h(x)$: Tree에 학습률 η를 곱해 예측값에 더함
  ⑧ Validation loss 확인 및 early stopping 체크
  ⑨ 대칭 트리들의 앙상블로 최종 모형, 추론 시 lookup table 방식으로 빠르게 예측

5. Ensemble Model: CatBoost

항목	LightGBM	XGBoost	CatBoost
개발사	Microsoft	DMLC	Yandex
출시연도	2017	2016	2017
학습 속도	⭐⭐⭐⭐⭐	⭐⭐⭐	⭐⭐⭐⭐
예측 속도	⭐⭐⭐⭐	⭐⭐⭐	⭐⭐⭐⭐⭐
정확도	⭐⭐⭐⭐	⭐⭐⭐⭐⭐	⭐⭐⭐⭐
메모리 효율	⭐⭐⭐⭐⭐	⭐⭐⭐	⭐⭐⭐⭐
범주형 처리	보통	없음(수동)	자동(최강)
과적합 방지	중간	강함	매우 강함
사용 추천 상황	대용량 데이터 속도 우선	경진대회 정확도 우선	범주형 많은 데이터 간편한 사용

수백만 건 이상의 대규모 데이터셋
학습 속도가 중요한 프로덕션 환경
메모리 제약이 있는 서버
실시간 재학습이 필요한 파이프라인

"대규모 데이터에서 가장 빠른 학습"

안정적이고 검증된 알고리즘 선호
L1/L2 정규화로 세밀한 튜닝 필요
다양한 목적함수 커스터마이징

"경쟁 모델의 표준, 강력한 정규화 (정확성, 안정성) "

범주형 변수가 많은 데이터 (e.g. 추천)
전처리 시간을 최소화하고 싶을 때
빠른 추론이 필요한 서비스 배포

"전처리 없이 범주형 데이터 자동 처리"

6. Ensemble Model: isolation forest

- **IsolationForest**
  - 이상치 탐지(Anomaly Detection) 전용 알고리즘
  - 부스팅 계열과의 차이
    - "정상 데이터를 학습"하는 접근 대신, “이상치는 고립시키기 쉽다 " 는 접근
    - 정상 데이터: 밀집된 형태로, 많은 분기를 발생시킴
    - 이상치: 드물고 떨어져 있으며 적은 분기만으로도 고립, 분기 횟수가 적을수록 이상치 가능성

6. Ensemble Model: isolation forest

- **IsolationForest**
  - 장점
    - 비지도 학습으로 작동, 이상치 데이터를 별도로 수집할 필요가 없음
    - 학습과 예측 속도가 빠르고, 고차원 데이터에도 적용
    - 모든 특성 쌍의 거리를 계산하지 않아도 되어, 계산 비용이 낮음
    - 적은 하이퍼파라미터(트리 수, 서브샘플 크기, contamination)

  - 단점
    - 분기 기준 무작위: 이상치가 특정 feature 조합에서만 드러나면 탐지 어려움 (다변량 이상치)
    - contamination 값 (실제 이상치 비율) 을 사전에 설정
    - 군집 형태의 이상치(여러 개가 뭉쳐 있는 경우)는 정상으로 오분류 가능성
    - 결과에 대한 직관적 해석 불가하여 설명이 어려움
    - 데이터 분포가 균일에 가까울 경우 정상과 이상치의 경로 길이 차이가 줄어 성능이 저하

6. Ensemble Model: isolation forest

- **Isolation Forest 절차**

① 전체 데이터에서 무작위로 서브샘플(보통 256개) 추출

② 서브샘플에서 특성 하나를 무작위로 선택

③ 선택한 특성의 min, max 사이에서 분기 기준값을 무작위로 선택

④ 기준값을 기준으로 데이터를 좌/우로 분기

⑤ 각 노드에 샘플이 1개가 되거나 지정한 최대 깊이에 도달할 때까지 2~4를 반복하여 Isolation Tree 하나를 완성

⑥ 1~5를 반복하여 지정한 수(보통 100개)의 트리로 이루어진 Forest 구성

⑦ 예측 시 각 샘플이 모든 트리에서 고립되기까지 걸린 평균 경로 길이(path length)를 계산

⑧ 평균 경로 길이를 정규화하여 이상 점수(Anomaly Score) 변환

- 점수가 1에 가까울수록 이상치, 0.5에 가까울수록 정상

⑨ 지정한 임계값(contamination) 이상의 점수를 가진 샘플을 이상치로 판정

Industrial Data Science Lab

Contact:
won.sang.l@gmail.com
https://sites.google.com/view/idslab